

**Линейная регрессия. Решение задачи
одномерной линейной регрессии.
Целевая функция. Метод градиентного
спуска. Множественная линейная
регрессия**

Содержание:

Одномерная линейная регрессия

- Постановка задачи
- Параметры модели
- Целевая функция

Метод градиентного спуска в задаче линейной регрессии

- Метод градиентного спуска
- Решение задачи линейной регрессии методом градиентного спуска

Задача множественной линейной регрессии

- Модель множественной линейной регрессии
- Решение задачи множественной линейной регрессии

Замечания

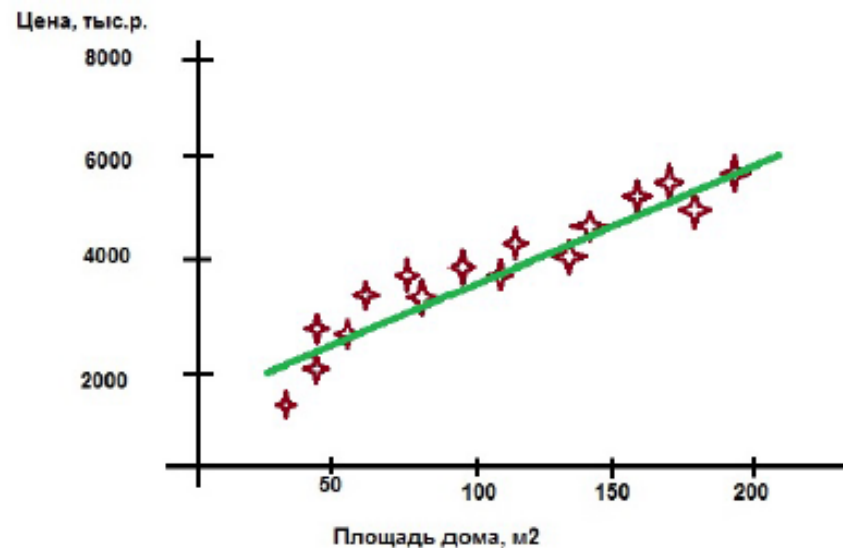
- Нормализация признаков
 - Поведение целевой функции
 - Выбор признаков и нелинейная регрессия
- 

Литература:

- Andrew Ng, Machine Learning Course (video, lecture notes, presentations, labs) <http://ml-class.org>
- Воронцов К.В. Машинное обучение (курс лекций) <http://www.machinelearning.ru>
- Гудфеллоу Я., Бенджио И., Курвилль А., Глубокое обучение
- Машинное обучение и анализ данных, курс Н.Ю. Золотых <http://www.uic.unn.ru/~zny/ml>

Задача прогнозирования цен на жильё

Имеются m данных о площади дома x и его цены y и предположение о линейной зависимости этих величин.

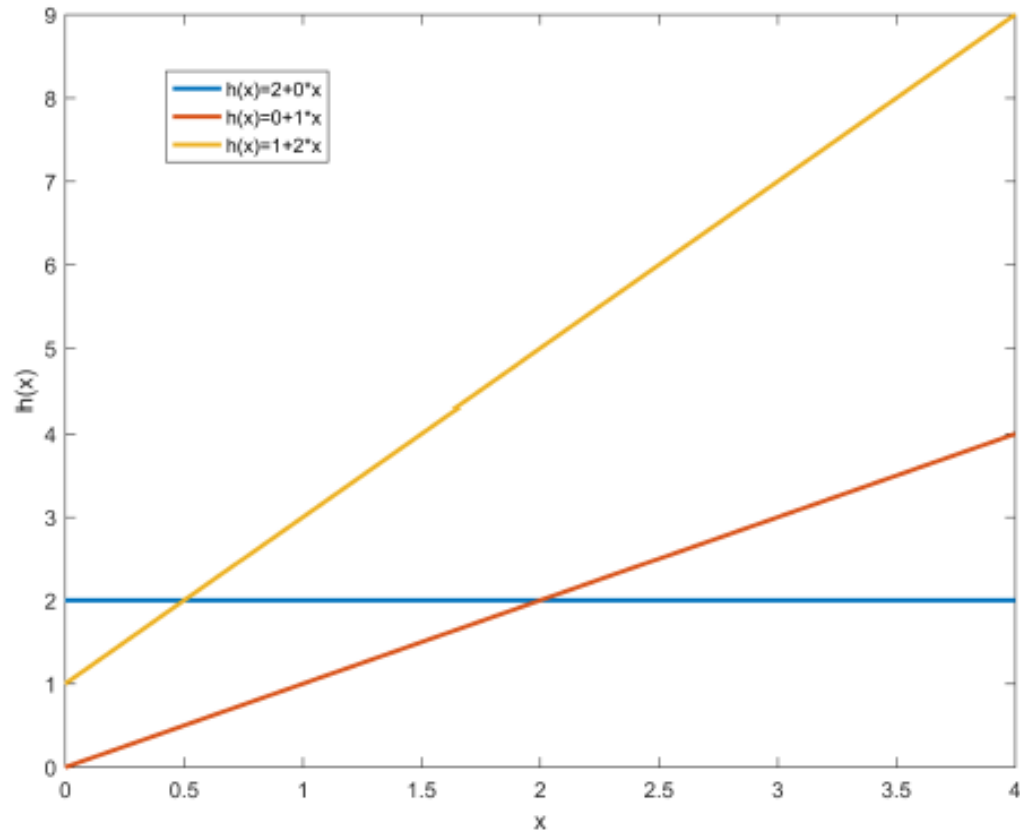


Требуется найти параметры θ_0, θ_1 в $h(x) = \theta_0 + \theta_1 x$ такие, что

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - h(x_i))^2 \rightarrow \min .$$

Параметры модели

θ_0, θ_1 определяют положение прямой $h(x) = \theta_0 + \theta_1 x$.



Как найти их оптимальные значения?

Целевая функция

Мы минимизируем функцию

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - h(x_i))^2.$$

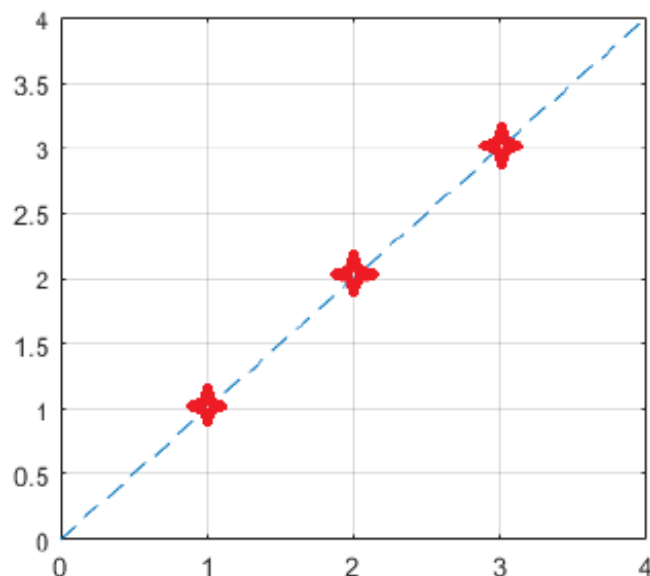
Поскольку $h(x_i) = \theta_0 + \theta_1 x_i$, то MSE будет зависеть от θ_0, θ_1 .

Итак, требуется найти минимум **целевой функции** двух переменных:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (y_i - h(x_i))^2.$$

Частный случай $\theta_0 = 0$

Упростим задачу. Пусть у нас будет всего один неизвестный параметр. Будем среди всех прямых вида $h(x) = \theta_1 x$, проходящих через начало координат, искать ту, которая проходит ближе всех к данным трем точкам.

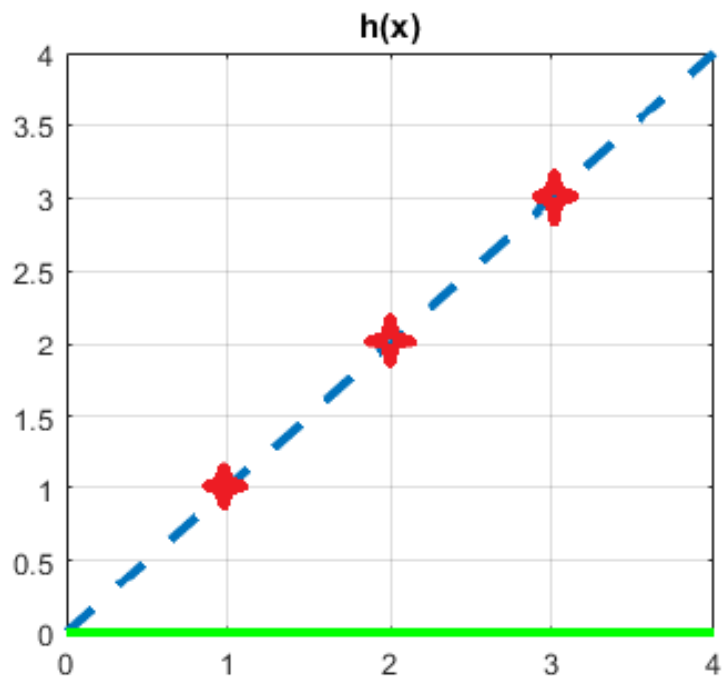


Минимизируем $J(\theta_1) = \frac{1}{2 \cdot 3} \sum_{i=1}^3 (y_i - h(x_i))^2$.

Частный случай $\theta_0 = 0$

Предположим, что мы взяли $\theta_1 = 0$. Тогда

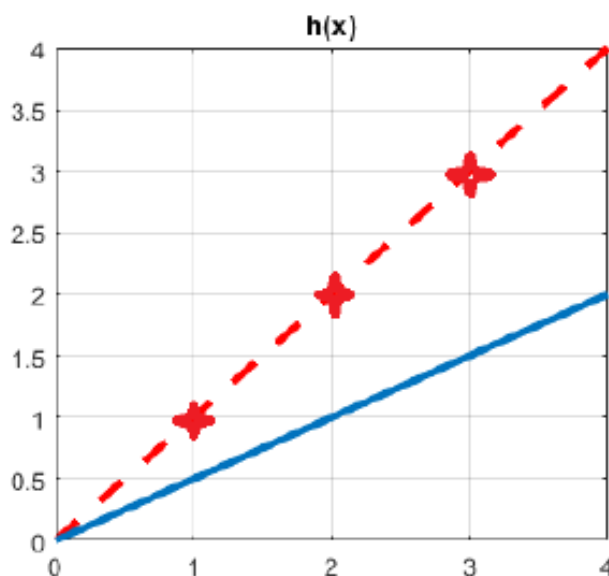
$$J(0) = \frac{1}{2 \cdot 3} ((1 - 0)^2 + (2 - 0)^2 + (3 - 0)^2) = \frac{14}{6} \approx 2.3.$$



Частный случай $\theta_0 = 0$

При $\theta_1 = 0.5$ значение $J(\theta_1)$ будет меньше:

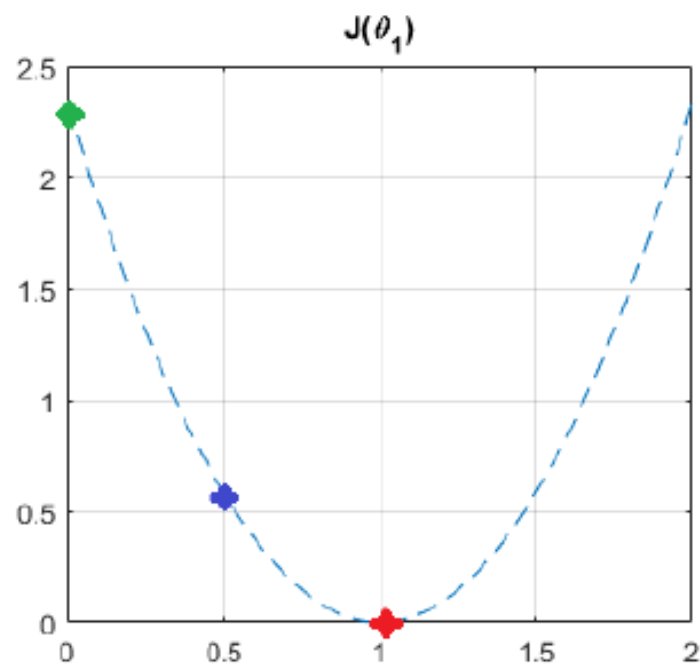
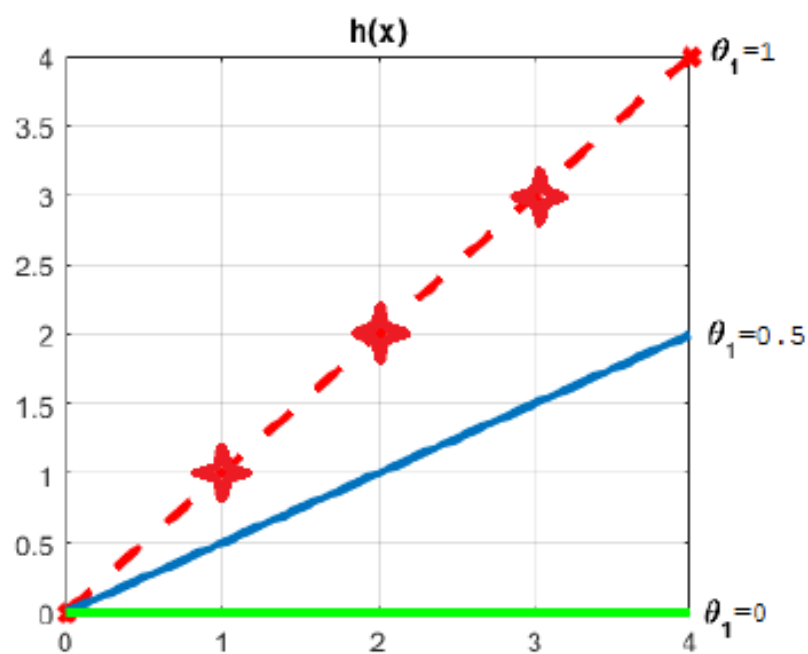
$$J(0.5) = \frac{1}{2 \cdot 3} ((1 - 0.5)^2 + (2 - 1)^2 + (3 - 1.5)^2) = \frac{14}{24} \approx 0.6.$$



«Двигая» прямую $h(x) = \theta_1 x$ за счет выбора θ_1 мы будем получать соответствующие значения $J(\theta_1)$.

Частный случай $\theta_0 = 0$

Ясно, что оптимальное положение прямой соответствует $\theta_1 = 1$ и дает значение $J(\theta_1) = 0$.



Частный случай $\theta_0 = 0$

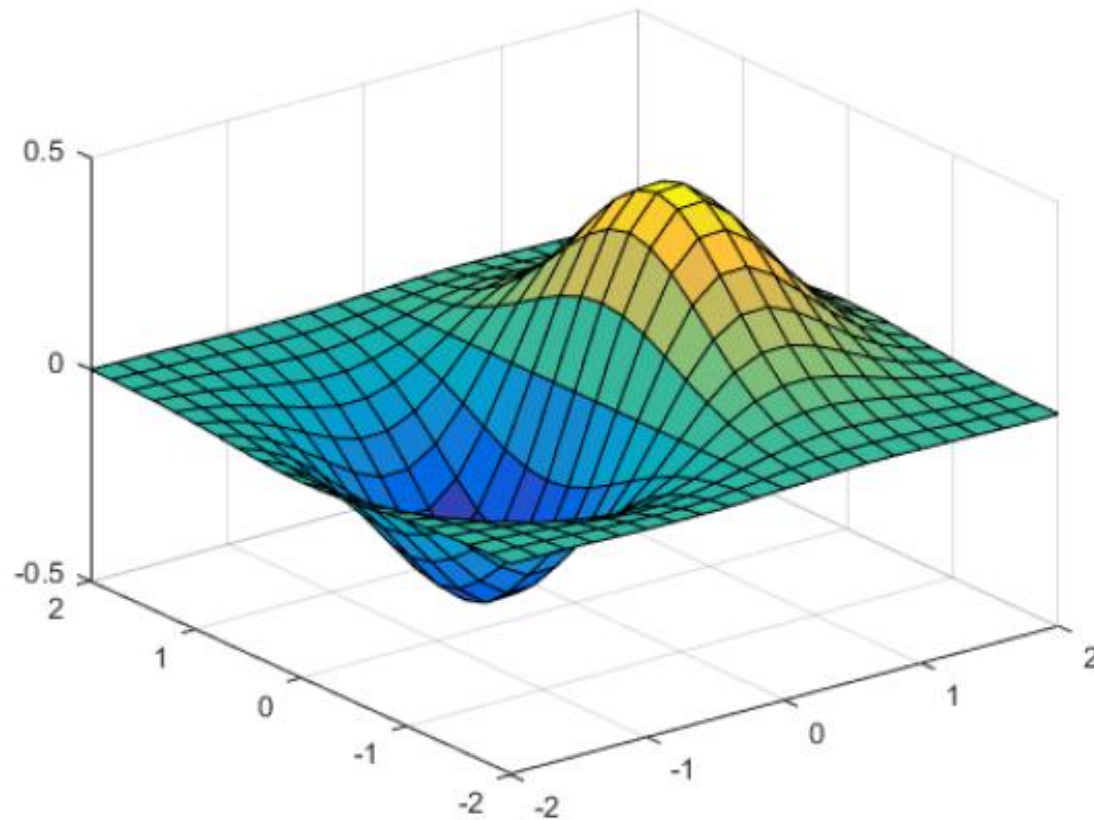
Несложно получить и явное выражение для целевой функции:

$$\begin{aligned} J(\theta_1) &= \frac{1}{2 \cdot 3} \sum_{i=1}^3 (y_i - h(x_i))^2 = \\ &= \frac{1}{2 \cdot 3} ((1 - \theta_1 \cdot 1)^2 + (2 - \theta_1 \cdot 2)^2 + (3 - \theta_1 \cdot 3)^2) = \\ &= \frac{14}{6} (1 - \theta_1)^2. \end{aligned}$$

Минимум такой функции, как и ожидалось, принимается в точке $\theta_1 = 1$.

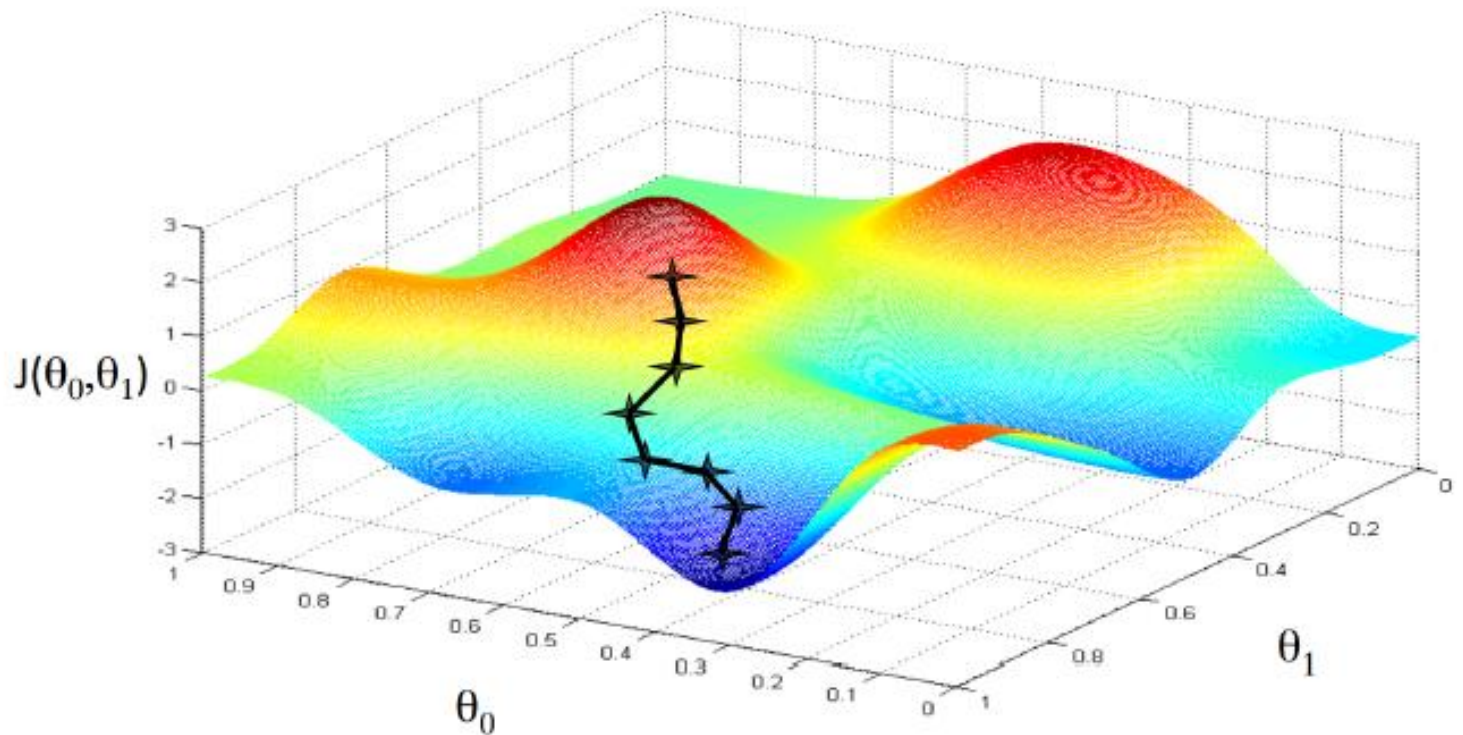
Целевая функция

В общем случае график целевой функции представляет собой сложную поверхность. Наша цель – прийти в точку минимума.



Метод градиентного спуска

Будем искать $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$ методом градиентного спуска, двигаясь из начальной точки в точку минимума.



Метод градиентного спуска

Алгоритм

Вход: целевая функция $J(\theta_0, \theta_1)$, начальная точка $(\hat{\theta}_0, \hat{\theta}_1)$, шаг обучения α .

Выход: координаты точки локального минимума функции J .

Для текущих значений координат $(\theta_0^{old}, \theta_1^{old})$ вычислять новые:

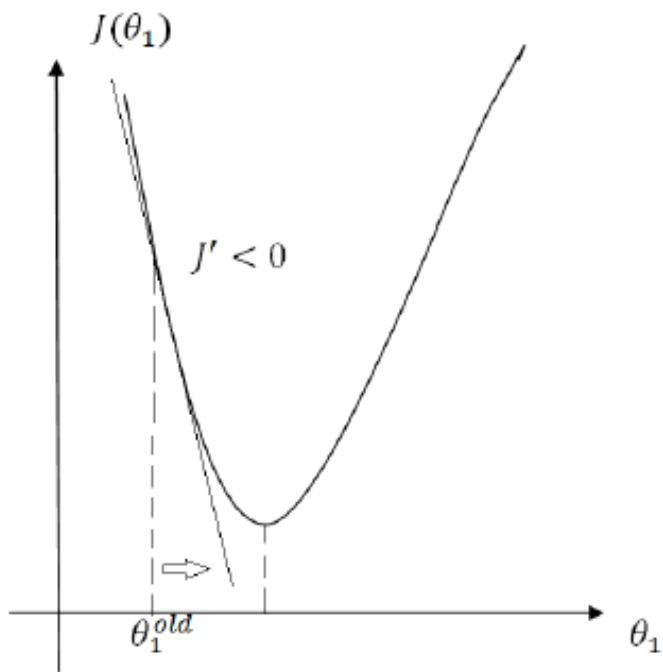
$$\theta_0^{new} = \theta_0^{old} - \alpha \left. \frac{\partial J}{\partial \theta_0} \right|_{(\theta_0^{old}, \theta_1^{old})}$$

$$\theta_1^{new} = \theta_1^{old} - \alpha \left. \frac{\partial J}{\partial \theta_1} \right|_{(\theta_0^{old}, \theta_1^{old})}$$

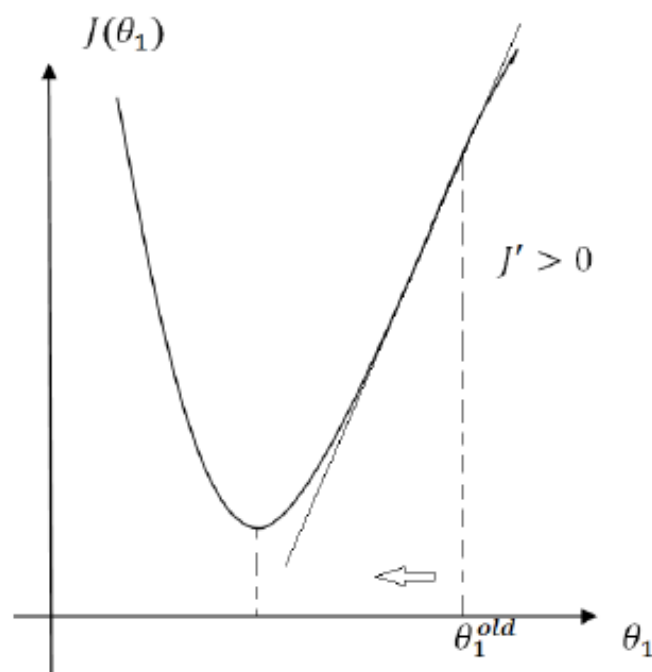
пока процесс не сойдется.

Частный случай $\theta_0 = 0$

Для интуитивного понимания метода вновь рассмотрим простой случай: $\theta_1^{new} = \theta_1^{old} - \alpha \left. \frac{dJ}{d\theta_1} \right|_{\theta_1^{old}}$, $\min_{\theta_1} J(\theta_1)$ —?



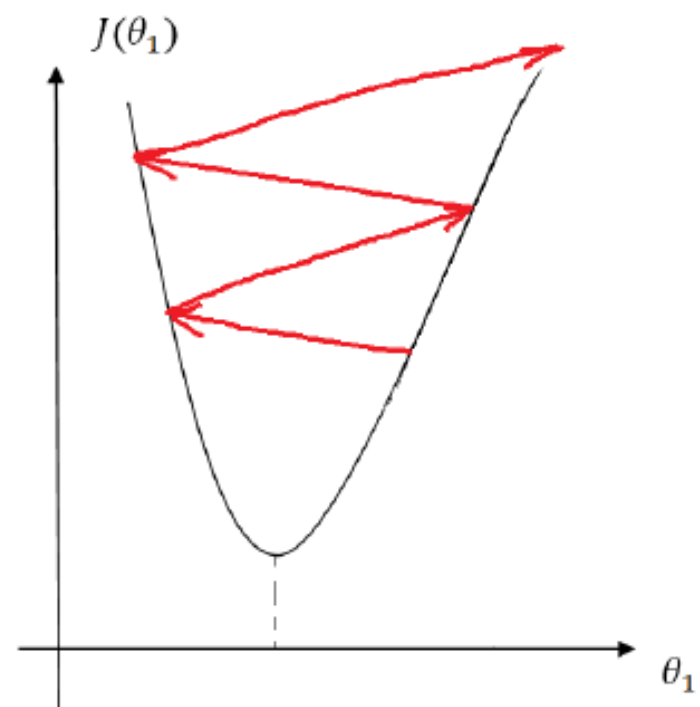
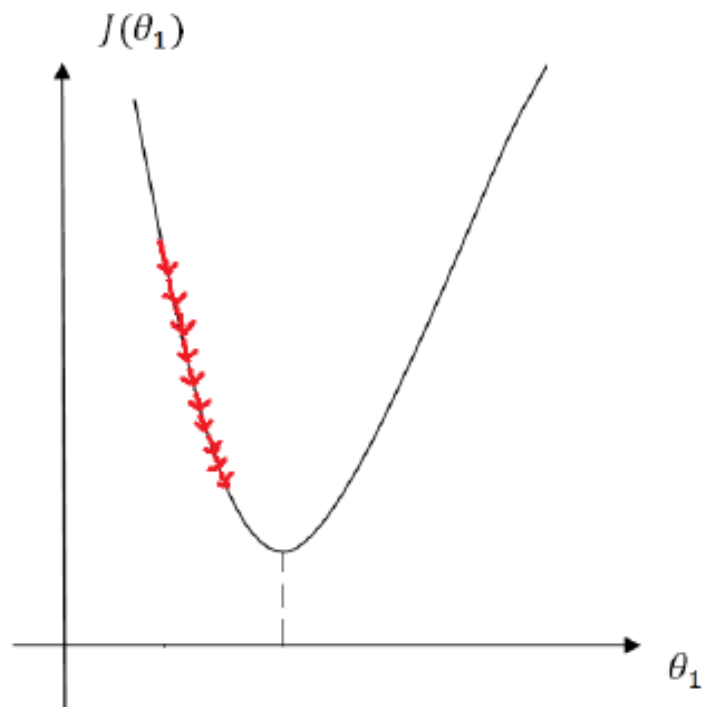
$$\theta_1^{new} > \theta_1^{old}$$



$$\theta_1^{new} < \theta_1^{old}$$

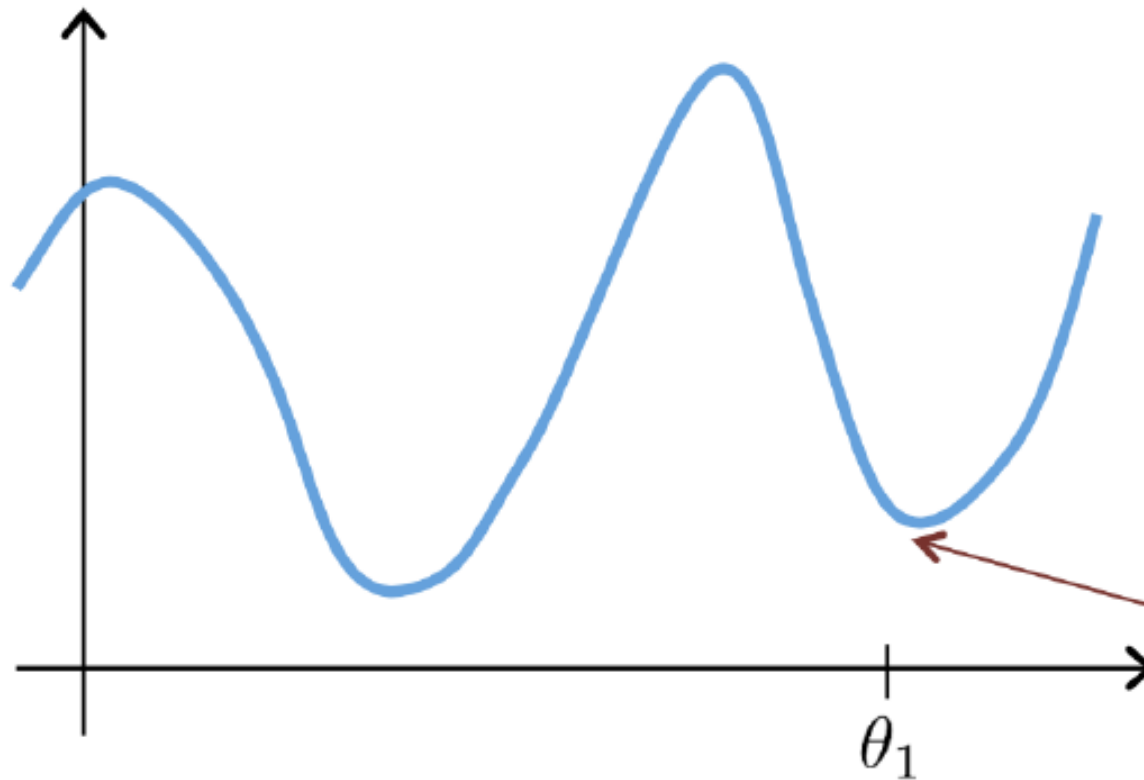
Шаг обучения

Если шаг обучения α мал, процесс идет очень медленно. Напротив, при слишком большом шаге мы можем «перепрыгнуть» через минимум.



Шаг обучения

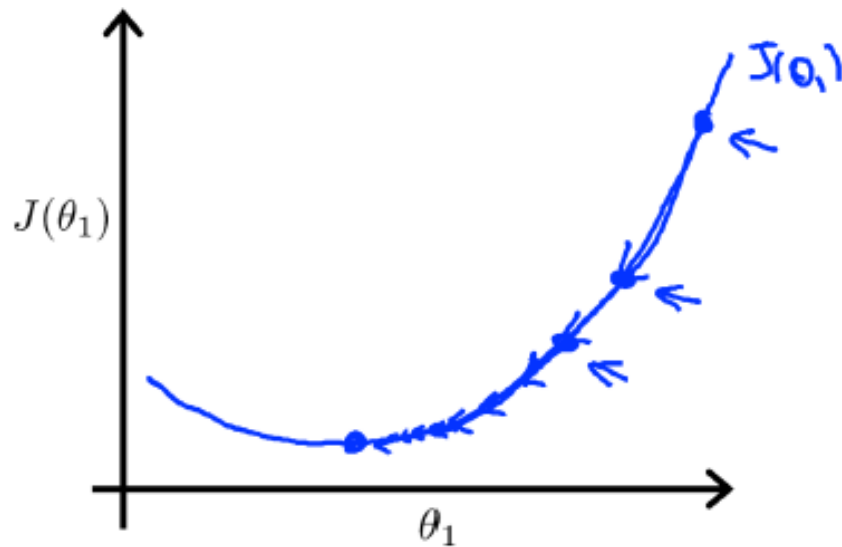
Кроме того, процесс может «застрять» в локальном минимуме.



Шаг обучения

При приближении к локальному минимуму мы автоматически начинаем делать все меньшие шаги (при неизменном α) в силу уменьшения значения производной.

$$\theta_1^{new} = \theta_1^{old} - \alpha \left. \frac{dJ}{d\theta_1} \right|_{\theta_1^{old}}$$



Решение задачи линейной регрессии методом градиентного спуска

Алгоритм градиентного спуска

$$\theta_0^{new} = \theta_0^{old} - \alpha \left. \frac{\partial J}{\partial \theta_0} \right|_{(\theta_0^{old}, \theta_1^{old})}$$

$$\theta_1^{new} = \theta_1^{old} - \alpha \left. \frac{\partial J}{\partial \theta_1} \right|_{(\theta_0^{old}, \theta_1^{old})}$$

Задача линейной регрессии

$$h(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h(x_i) - y_i)^2.$$

Решение задачи линейной регрессии методом градиентного спуска

Найдем частные производные функции:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2.$$

Получим:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i),$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i) \cdot x_i.$$

Решение задачи линейной регрессии методом градиентного спуска

Таким образом, в задаче линейной регрессии обновление параметров (θ_0, θ_1) производится по формулам:

$$\theta_0^{new} = \theta_0^{old} - \alpha \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i),$$

$$\theta_1^{new} = \theta_1^{old} - \alpha \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i) \cdot x_i.$$

Итерационный процесс останавливается, когда $J(\theta_0^{(n)}, \theta_1^{(n)}) \approx 0$ и/или $\|\theta_i^{(n+1)} - \theta_i^{(n)}\| \approx 0$.

Задача множественной линейной регрессии

Как правило, интересующая нас величина зависит не от одной, а от многих переменных:

Площадь, x_1	Число спален, x_2	Этаж, x_3	Возраст, x_4	Цена, y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

и, вместо модели одномерной регрессии:

$$h(x) = \theta_0 + \theta_1 x$$

мы приходим к модели множественной линейной регрессии:

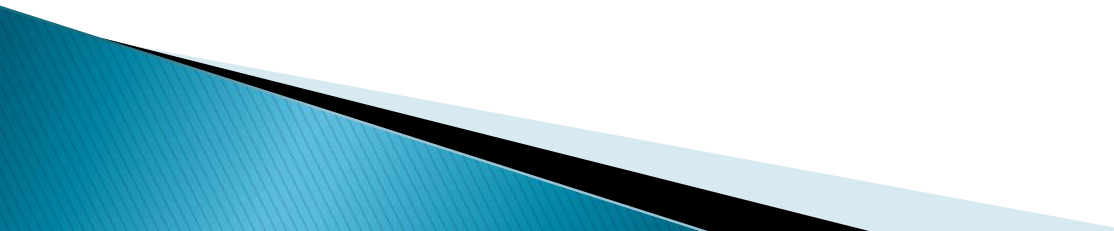
$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4.$$

Целевая функция в задаче множественной линейной регрессии

В случае множественной линейной регрессии значение целевой функции для набора данных $x^{(i)}, y^{(i)}, i = 1, \dots, m$ находится по формуле:

$$J(\theta) = J(\theta_0, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2.$$

Выглядит, как в одномерном случае!



Метод градиентного спуска в задаче множественной линейной регрессии

Обновление параметров $\theta_0, \dots, \theta_n$ производится по похожим формулам:

$$\theta_0^{new} = \theta_0^{old} - \alpha \frac{1}{m} \sum_{i=1}^m \left(h(x^{(i)}) - y^{(i)} \right),$$

$$\theta_1^{new} = \theta_1^{old} - \alpha \frac{1}{m} \sum_{i=1}^m \left(h(x^{(i)}) - y^{(i)} \right) \cdot x_1^{(i)}.$$

...

$$\theta_n^{new} = \theta_n^{old} - \alpha \frac{1}{m} \sum_{i=1}^m \left(h(x^{(i)}) - y^{(i)} \right) \cdot x_n^{(i)}.$$

Замечание 1: Нормализация признаков

Предположим, в нашей задаче цена дома зависит от двух признаков:

- x_1 : площадь дома, фут², принимает значение в диапазоне 0 — 2000,
- x_2 : число спален, шт., принимает значение в диапазоне 1 — 5.

Из-за такой разницы в масштабе переменных x_1, x_2 изменение значения целевой функции $J(\theta)$ (поскольку $h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$) будет очень значительным даже при небольшом изменении θ_1 (из-за все равно большого слагаемого $\theta_1 x_1$) и напротив, небольшим при изменении θ_2 .

Методу градиентного спуска трудно работать в таких условиях.

Нормализация признаков

Мы можем улучшить скорость сходимости метода градиентного спуска, если вместо старых «несоизмеримых» переменных x_1, x_2 рассмотрим, например, такие:

$$x_1 = \frac{x_1}{2000}, \quad x_2 = \frac{x_2}{5}.$$

Новые x_1, x_2 меняются примерно в одном диапазоне и это очень хорошо.

Нормализация признаков

Широко распространенными являются следующие способы нормализации:

- Минимакс – линейное преобразование данных в диапазоне $[0, 1]$, где минимальное и максимальное масштабируемые значения соответствуют 0 и 1 соответственно:

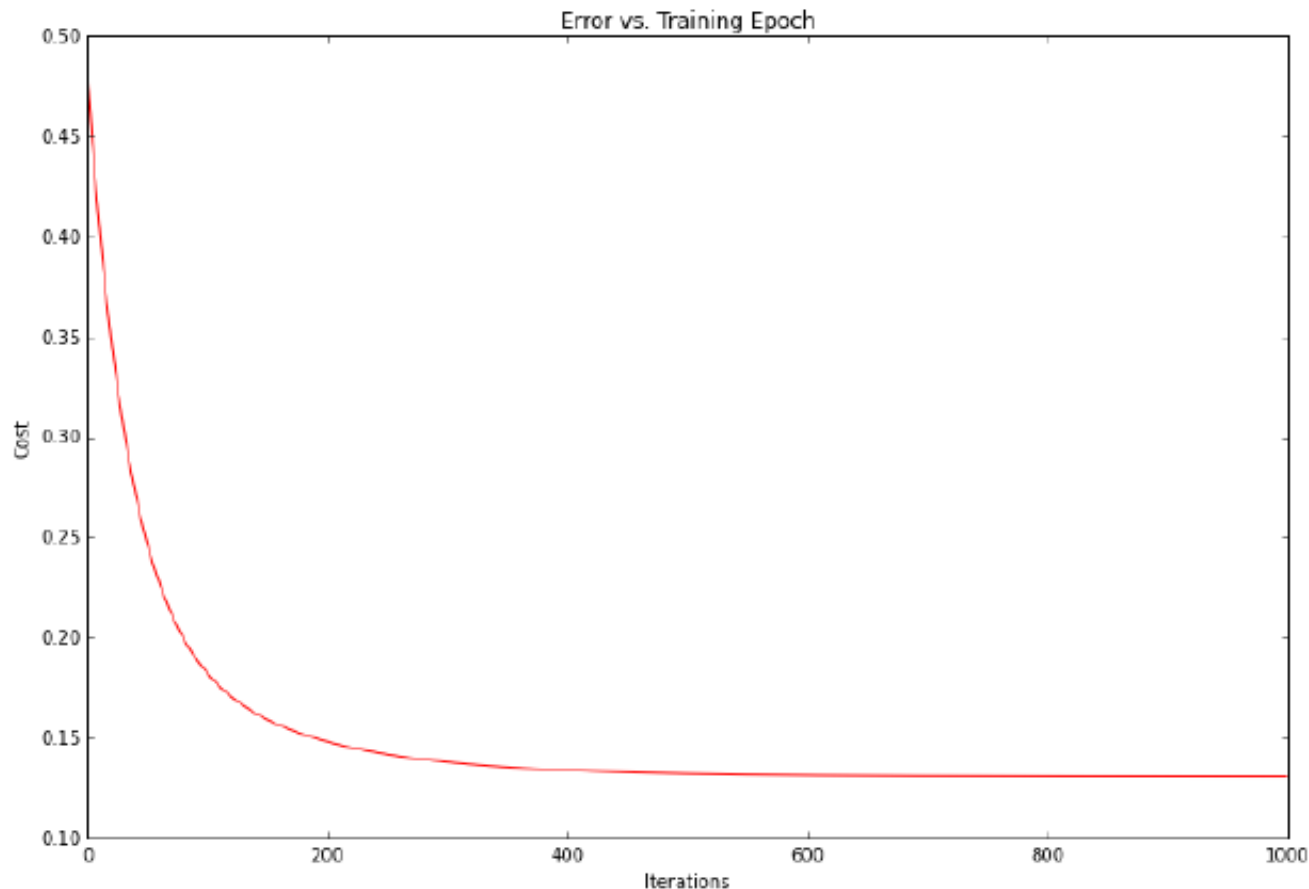
$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}},$$

- Z-масштабирование данных на основе среднего значения и стандартного отклонения – деление разницы между переменной и средним значением на стандартное отклонение:

$$z = \frac{X - \mu}{\sigma}$$

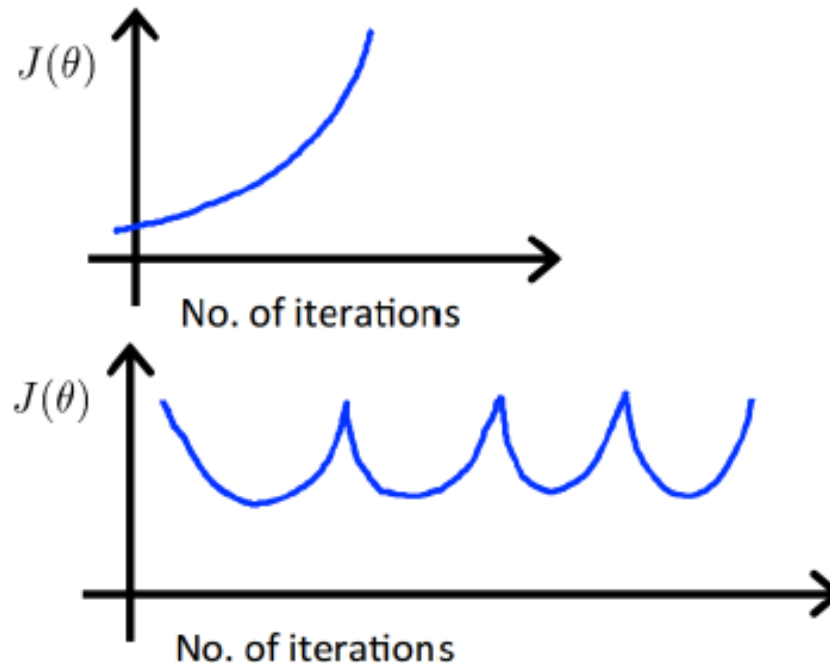
Замечание 2: Следим за поведением целевой функции

При правильной работе метода градиентного спуска значения $J(\theta)$ уменьшаются с ростом числа эпох обучения:



Замечание 2: Следим за поведением целевой функции

Если Вы видите что-то подобное:



нужно уменьшить шаг обучения α (либо найти ошибку в коде).

Замечание 3: Выбор признаков



Вы можете конструировать свои признаки. Например, Вы могли бы рассматривать длину x_1 и ширину x_2 Вашего дома в качестве двух входных признаков в вашей линейной модели:

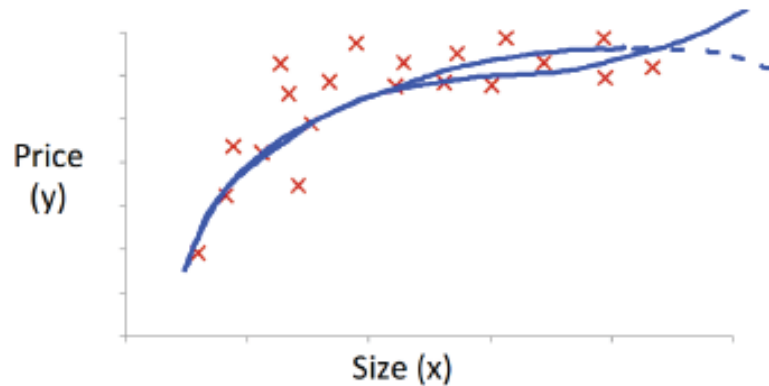
$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2.$$

А можете взять новый признак – площадь $x = x_1 \cdot x_2$ и модель:

$$h(x) = \theta_0 + \theta_1 x.$$

Замечание 4: Нелинейная регрессия

Все, что мы рассмотрели обобщается и на случай нелинейной зависимости.



Например,
в таком случае нам хочется
использовать функцию:

$$h(x) = \theta_0 + \theta_1 x + \theta_2 x^2,$$

$$h(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3,$$

а может быть тут хорошо подойдет такая модель:

$$h(x) = \theta_0 + \theta_1 x + \theta_2 \sqrt{x}.$$

Нелинейная регрессия

Вводим новые признаки:

$$h(x) = \theta_0 + \theta_1 \overset{x_1}{\circlearrowleft x} + \theta_2 \overset{x_2}{\circlearrowleft x^2}$$

$$h(x) = \theta_0 + \theta_1 \overset{x_1}{\circlearrowleft x} + \theta_2 \overset{x_2}{\circlearrowleft x^2} + \theta_3 \overset{x_3}{\circlearrowleft x^3}$$

$$h(x) = \theta_0 + \theta_1 \overset{x_1}{\circlearrowleft x} + \theta_2 \overset{x_2}{\circlearrowleft \sqrt{x}}$$

и не забываем про нормализацию!

Замечание 5: Нормальное уравнение

Нормальное уравнение (Normal equation) – другой способ решения задачи линейной регрессии.

Для набора данных $x^{(i)}, y^{(i)}, i = 1, \dots, m$ значение целевой функции находится по формуле:

$$J(\theta) = J(\theta_0, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2.$$

Чтобы найти значения параметров $\theta_0, \dots, \theta_n$, доставляющих минимум этой функции, можно решить систему уравнений:

$$\frac{\partial}{\partial \theta_j} J(\theta) = 0, \quad j = 0, \dots, n,$$

которая имеет следующее решение: $\theta = (X^T X)^{-1} X^T y$, где $X_{m \times (n+1)}$, $y_{m \times 1}$, $\theta_{(n+1) \times 1}$.

Нормальное уравнение vs Градиентный спуск

Градиентный спуск

- Нужно выбирать α , т.е. возможно нужно запускать алгоритм несколько раз.
- Нужно следить, как идет итерационный процесс.
- Часто нужна нормализация данных.
- Работает хорошо, когда n большое.

Нормальное уравнение

- Не нужно выбирать α .
- Не итерационный алгоритм.
- Не нужна нормализация данных.
- Нужно вычислять $(X^T X)^{-1}_{(n+1) \times (n+1)}$.
- Работает медленно при больших n .

Нормальное уравнение vs Градиентный спуск

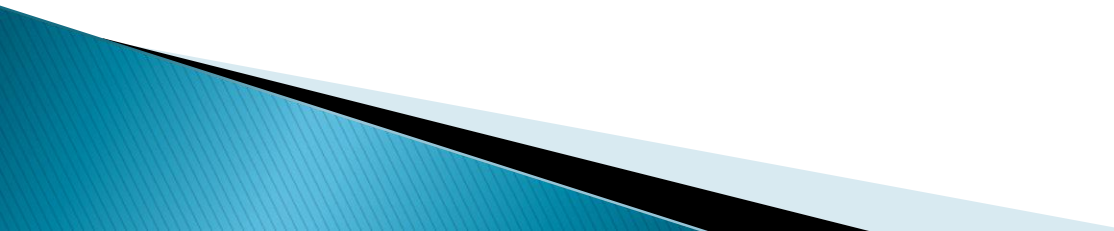
Градиентный спуск

- Нужно выбирать α , т.е. возможно нужно запускать алгоритм несколько раз.
- Нужно следить, как идет итерационный процесс.
- Часто нужна нормализация данных.
- Работает хорошо, когда n большое.

Нормальное уравнение

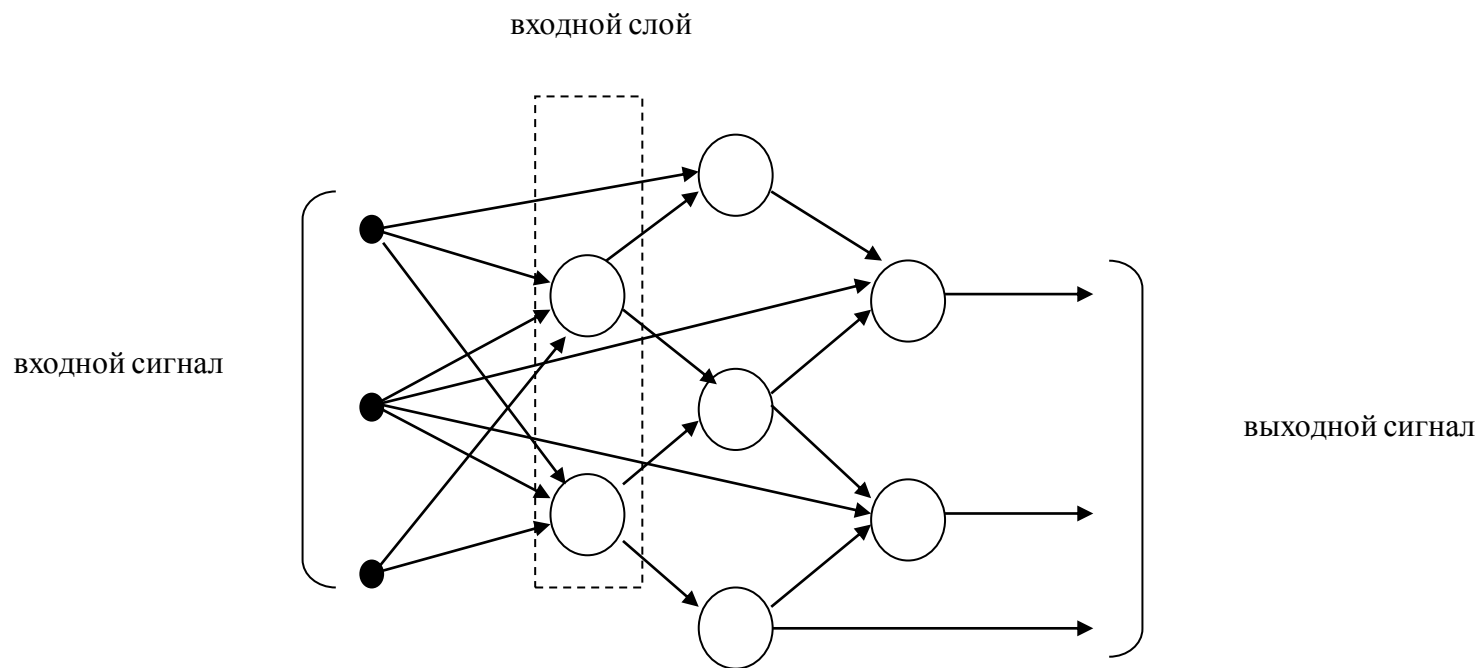
- Не нужно выбирать α .
- Не итерационный алгоритм.
- Не нужна нормализация данных.
- Нужно вычислять $(X^T X)^{-1}_{(n+1) \times (n+1)}$.
- Работает медленно при больших n .

Выводы

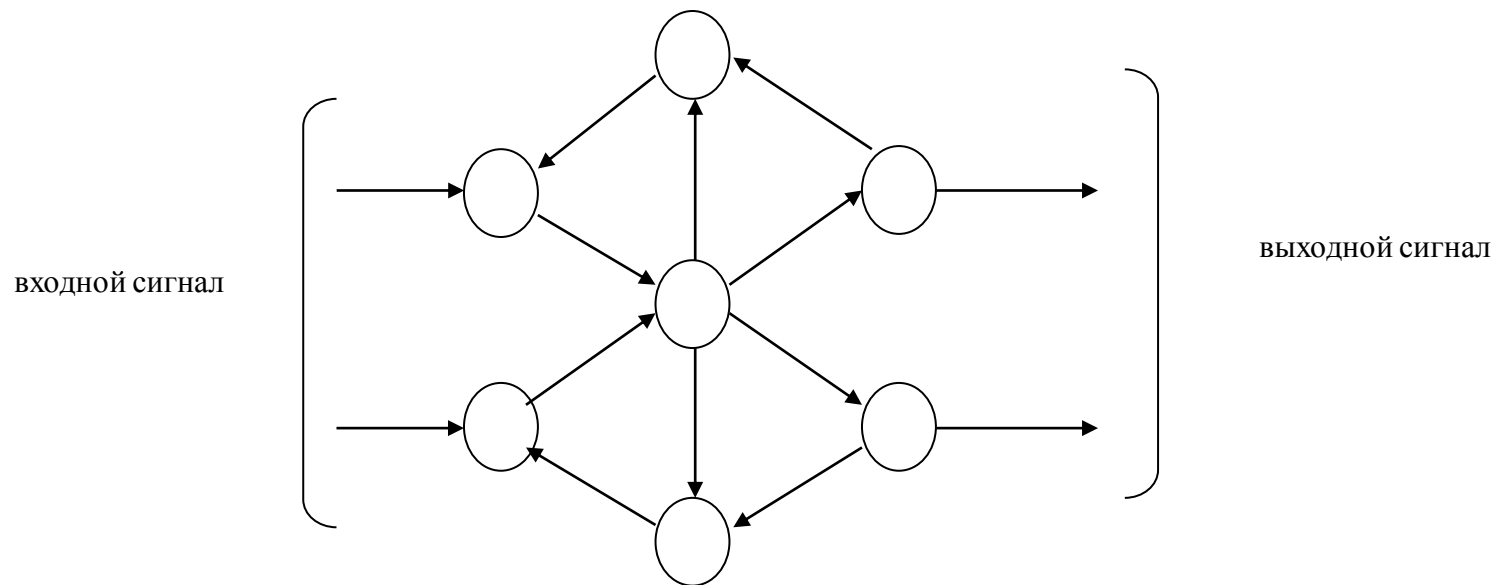
- Линейная регрессия – простая модель машинного обучения, параметры которой можно искать методом градиентного спуска. Этот же метод работает и в сложных нейронных сетях
 - Процесс обучения модели заключается в подборе параметров модели с целью минимизации целевой функции
 - За счет выбора признаков можно методом линейной регрессии построить и нелинейную модель
 - Почти для всех моделей машинного обучения нужна предварительная нормализация признаков
- 

Категории сетей:

- ▶ Слоистые сети без обратных связей (сети прямого распространения)

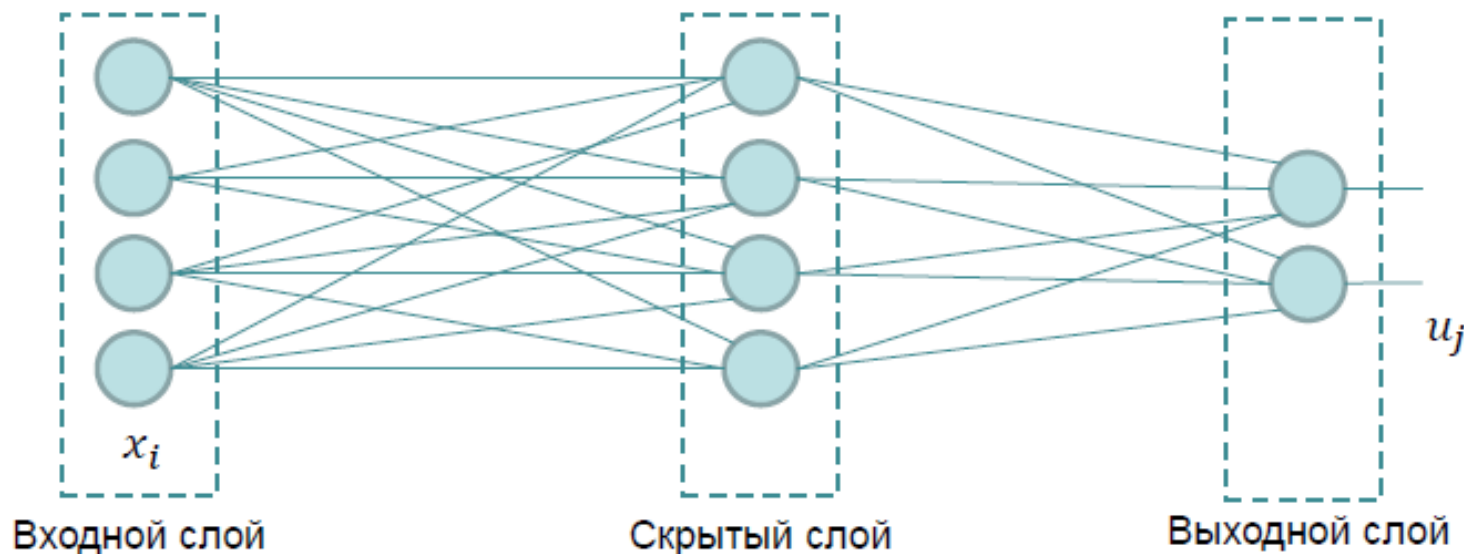


- Сети с обратными связями



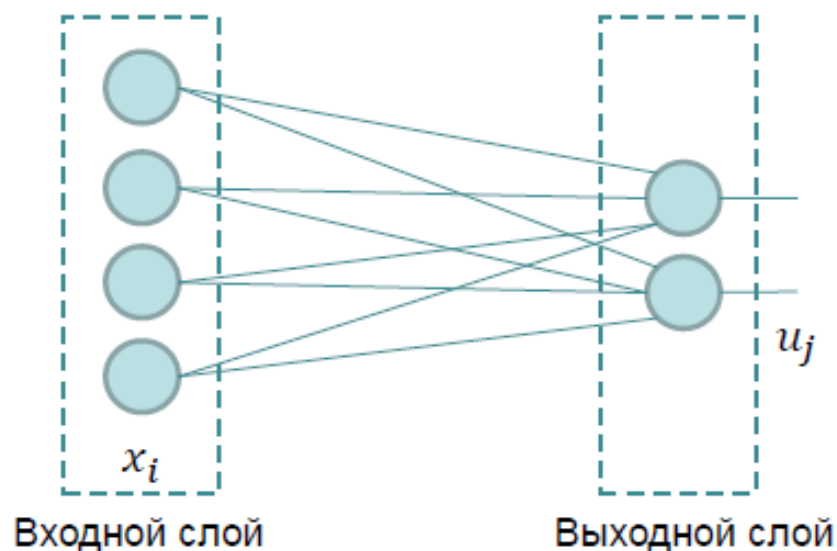
Общая структура модели многослойной полностью связанной нейронной сети (1)

- Многослойная нейронная сеть содержит нейроны, которые распределены по **СЛОЯМ**



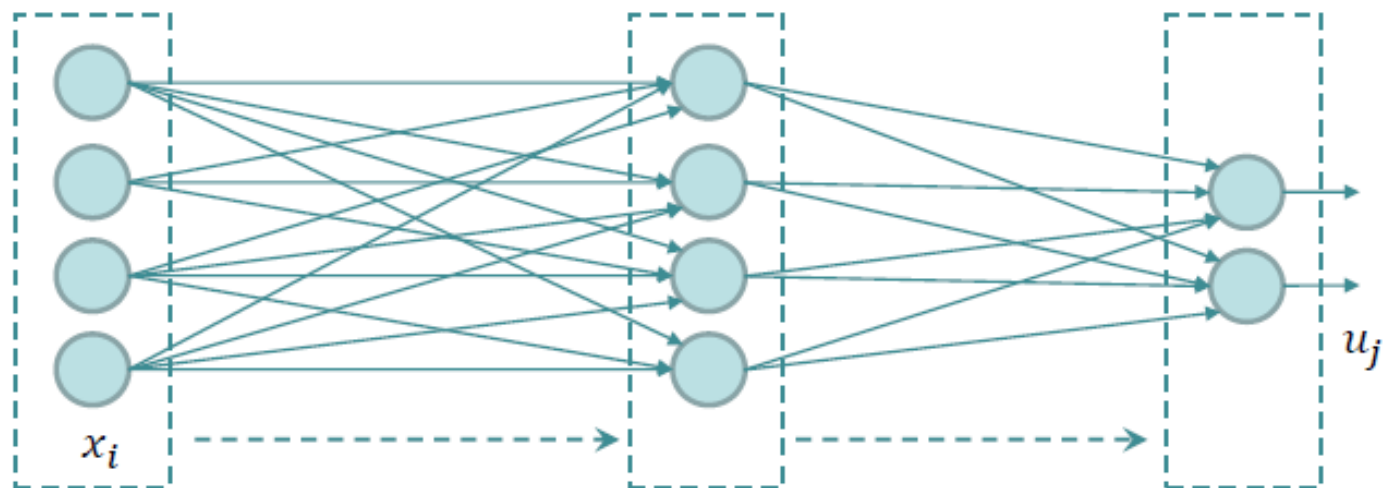
Общая структура модели многослойной полностью связанной нейронной сети (2)

- В простейшем случае в сети существует **входной и выходной слой**, и сеть называется **однослойной**



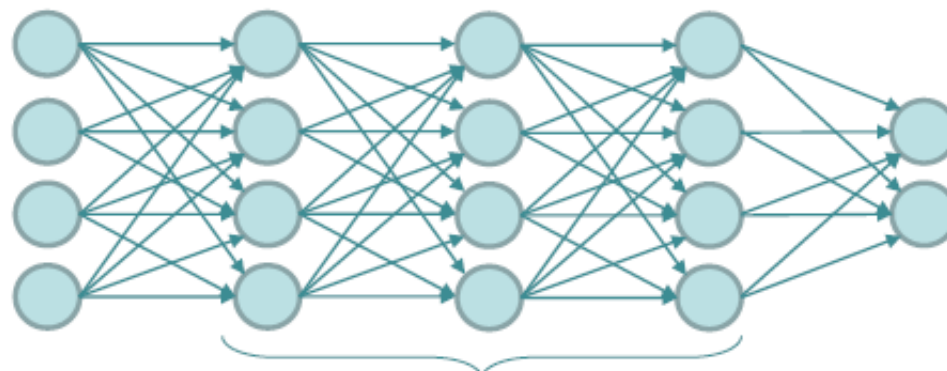
Общая структура модели многослойной полностью связанной нейронной сети (3)

- Если сигнал проходит от нейронов входного слоя к нейронам выходного, то такая сеть называется **сетью прямого распространения**



Общая структура модели многослойной полностью связанной нейронной сети (4)

- ❑ Сеть может содержать множество скрытых слоев называется **многослойной**
- ❑ Если все узлы слоя соединены с узлами следующего слоя, то слой называется **полностью связным**
- ❑ Если данное условие выполнено для всех слоев сети, то сеть называется **полностью связной** или **полносвязной** (Fully-Connected Neural Network, FCNN)



Входной слой

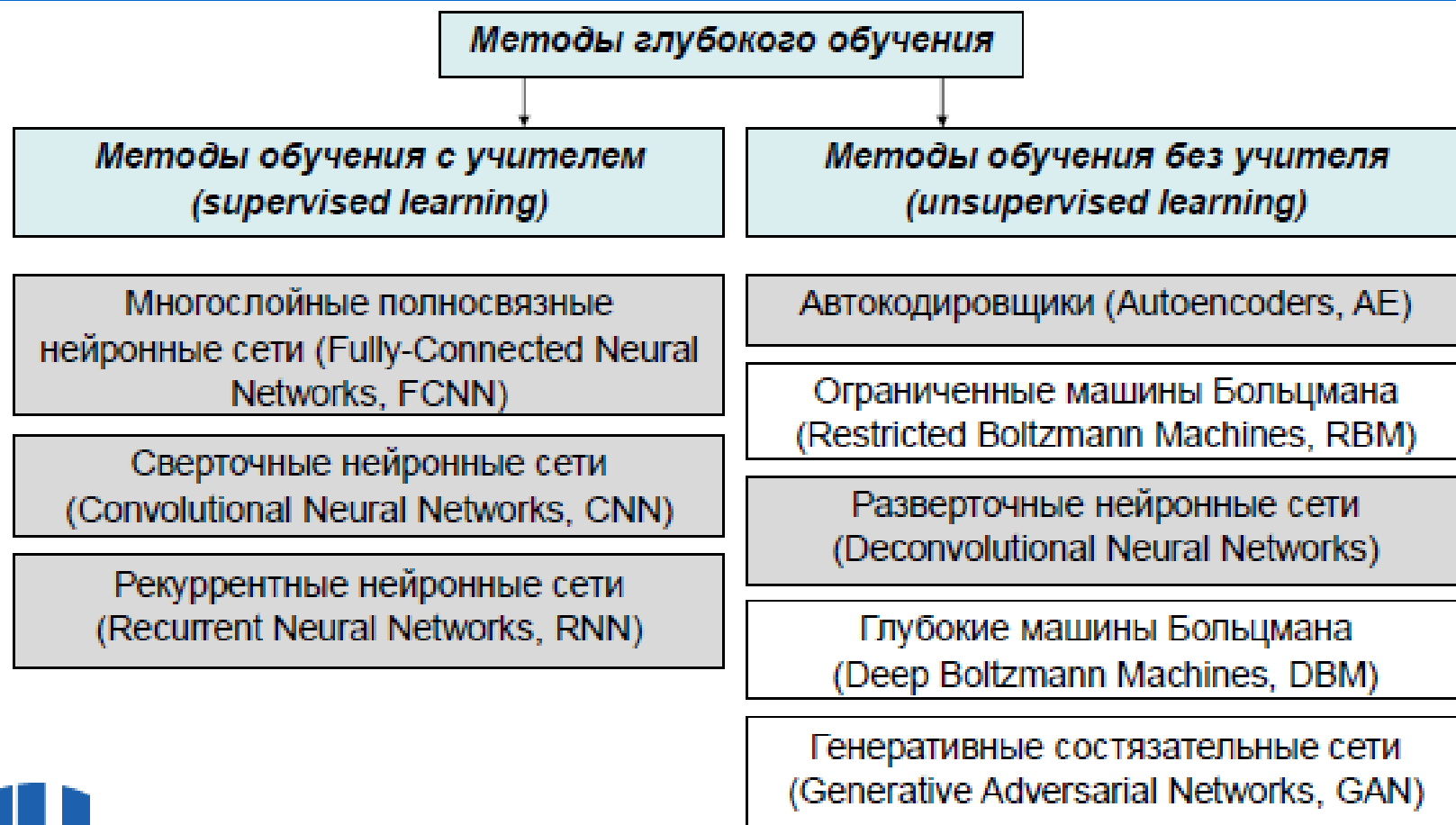
Скрытые слои

Выходной слой

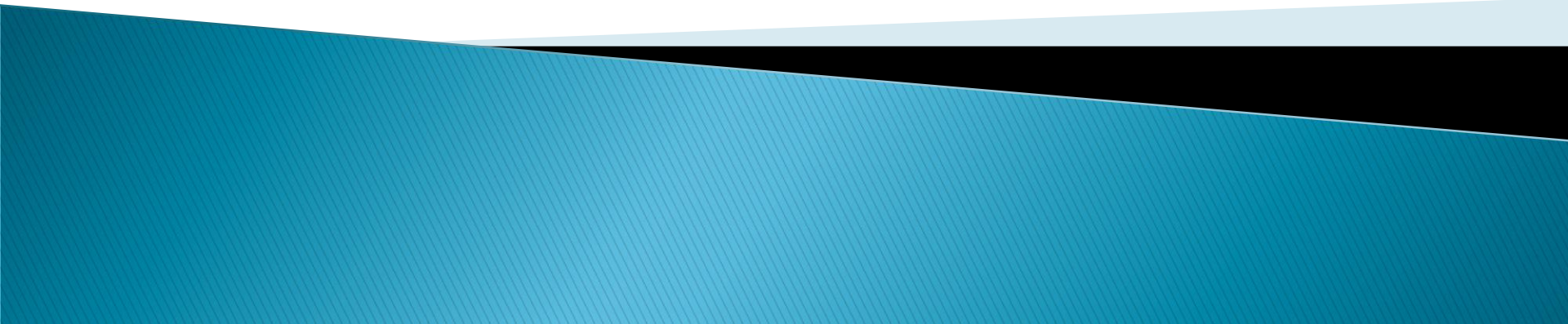
Задача обучения нейронной сети

- ❑ **Цель обучения** – подобрать веса нейронной сети наилучшим образом
- ❑ **Задача обучения** – задача минимизации **функции ошибки** или **функция стоимости** (cost function), отражающей разницу ожидаемого сигнала, полученного на выходе сети и фактического сигнала, соответствующего текущему входу, по всей обучающей выборке

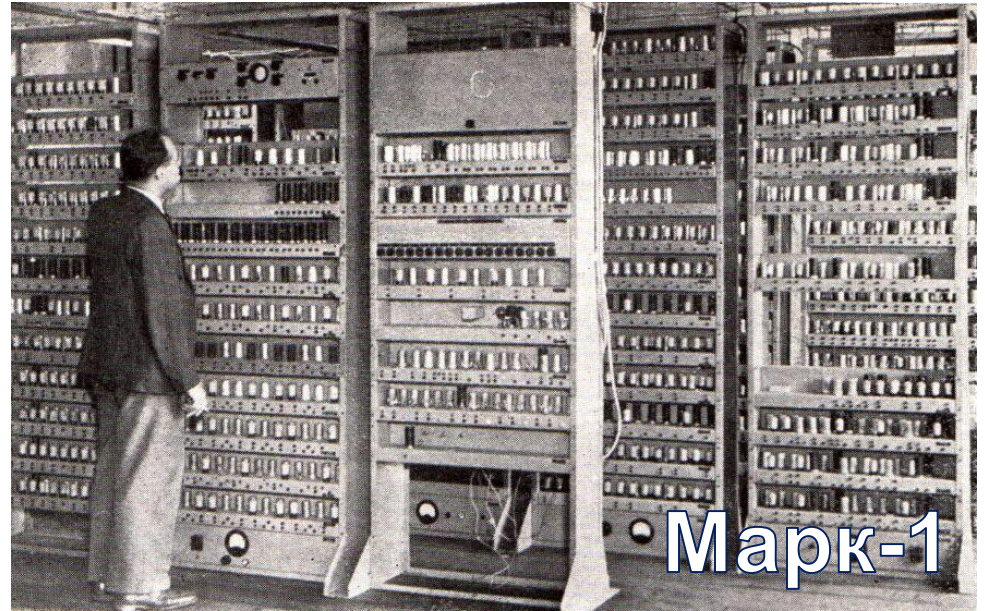
Классификация моделей по способу обучения. Далее в курсе подробно рассматриваются...



II. Перцептрон Розенблатта



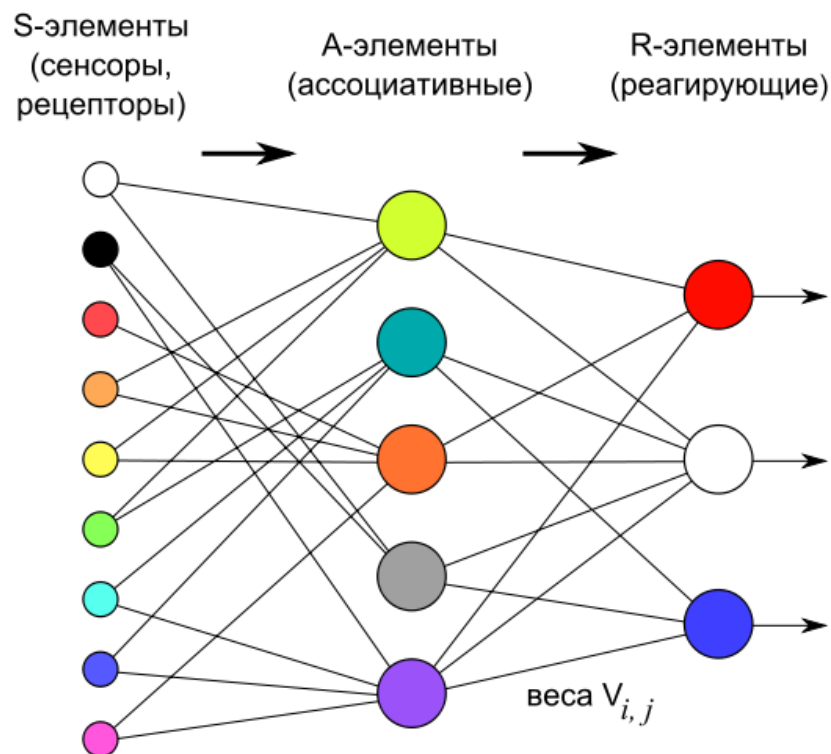
Персептрон Розенблатта



Перцептрон — математическая или компьютерная модель восприятия информации мозгом, предложенная Фрэнком Розенблаттом в 1957 году и впервые реализованная в виде электронной машины «Марк-1» в 1960 году.

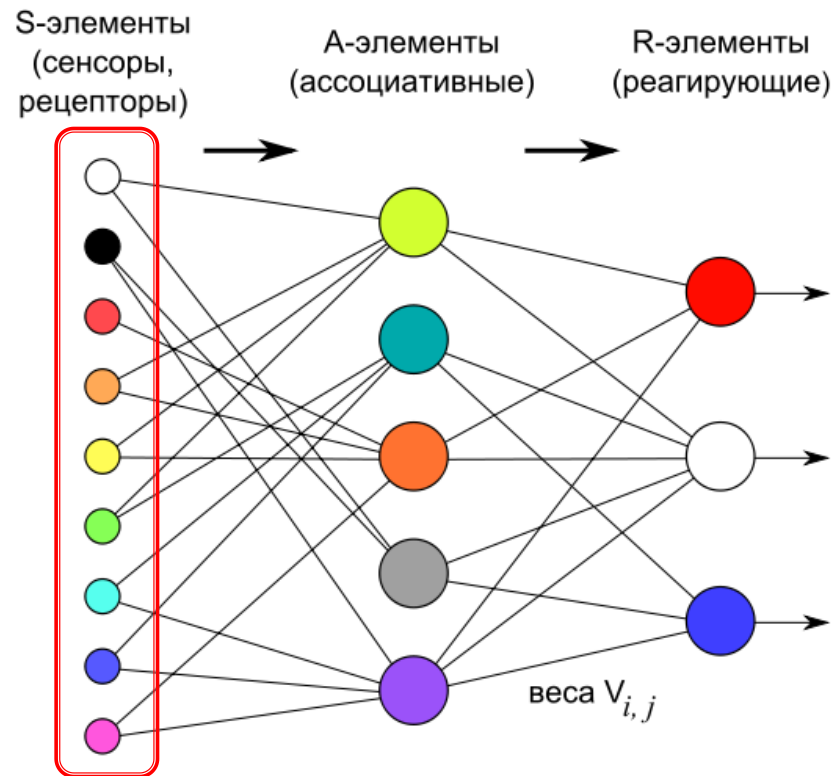
Перцептрон стал одной из первых моделей *нейросетей*, а «Марк-1» — первым в мире *нейрокомпьютером*.

Персептрон Розенблатта



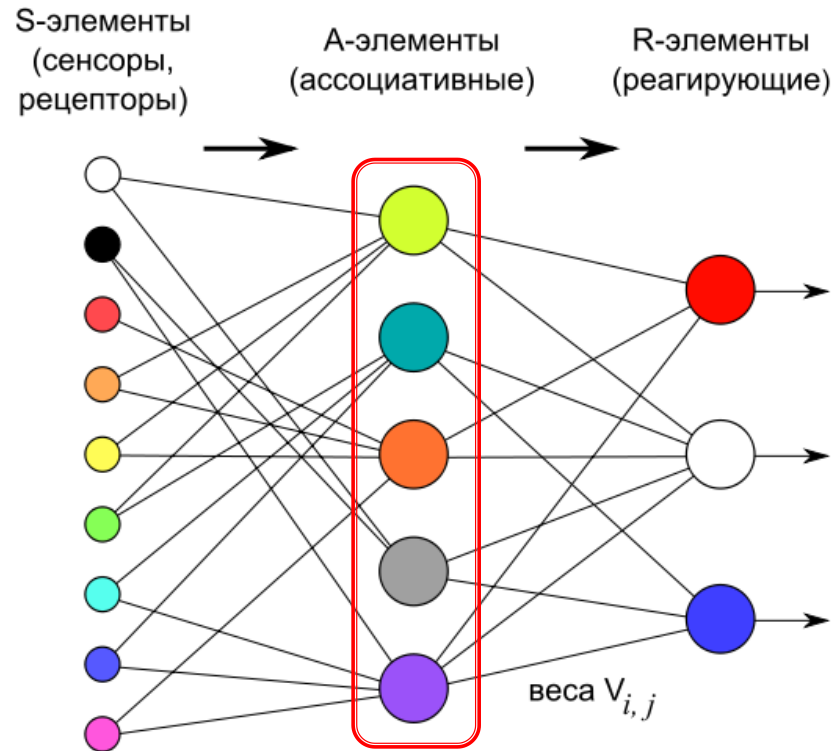
Персептрон представляет собой сеть, состоящую из S, A, R — элементов, с переменной матрицей взаимодействия V (элементы которой v_{ij} - весовые коэффициенты), определяемой последовательностью прошлых состояний активности сети.

Персептрон Розенблатта



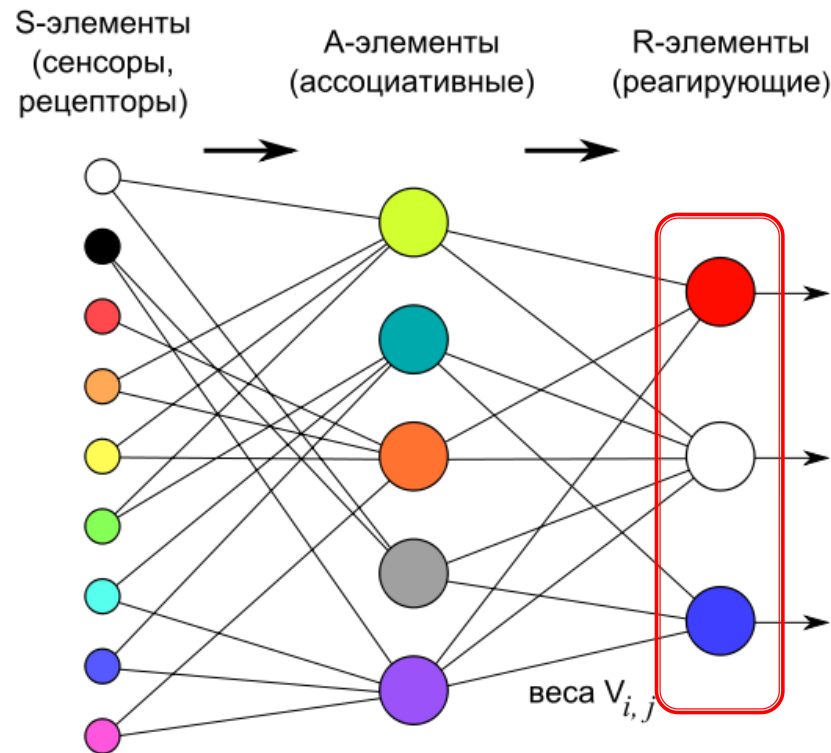
Простым S-элементом (Сенсорным элементом) является чувствительный элемент, который от воздействия какого-либо из видов энергии (например, света, звука, давления, тепла и т. п.) вырабатывает сигнал. Если входной сигнал превышает некоторый порог, то элемент выдаёт выходной сигнал +1, в противном случае выходной сигнал равен нулю.

Персептрон Розенблатта



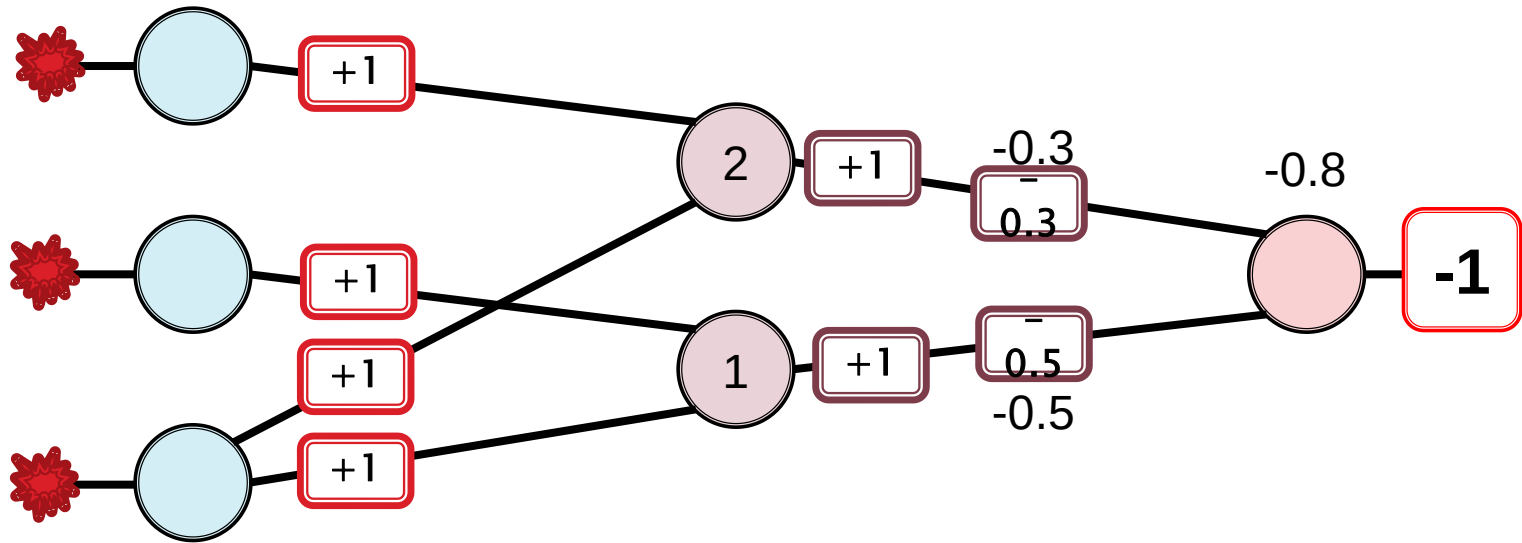
Простым А-элементом (Ассоциативным элементом) называется логический решающий элемент, который выдаёт выходной сигнал, когда алгебраическая сумма его входных сигналов равна или превышает некоторую пороговую величину. Выходной сигнал равен +1, если была превышена пороговая величина. В противном случае он равен нулю. Если +1, то говорят, что А-элемент является активным.

Персептрон Розенблатта



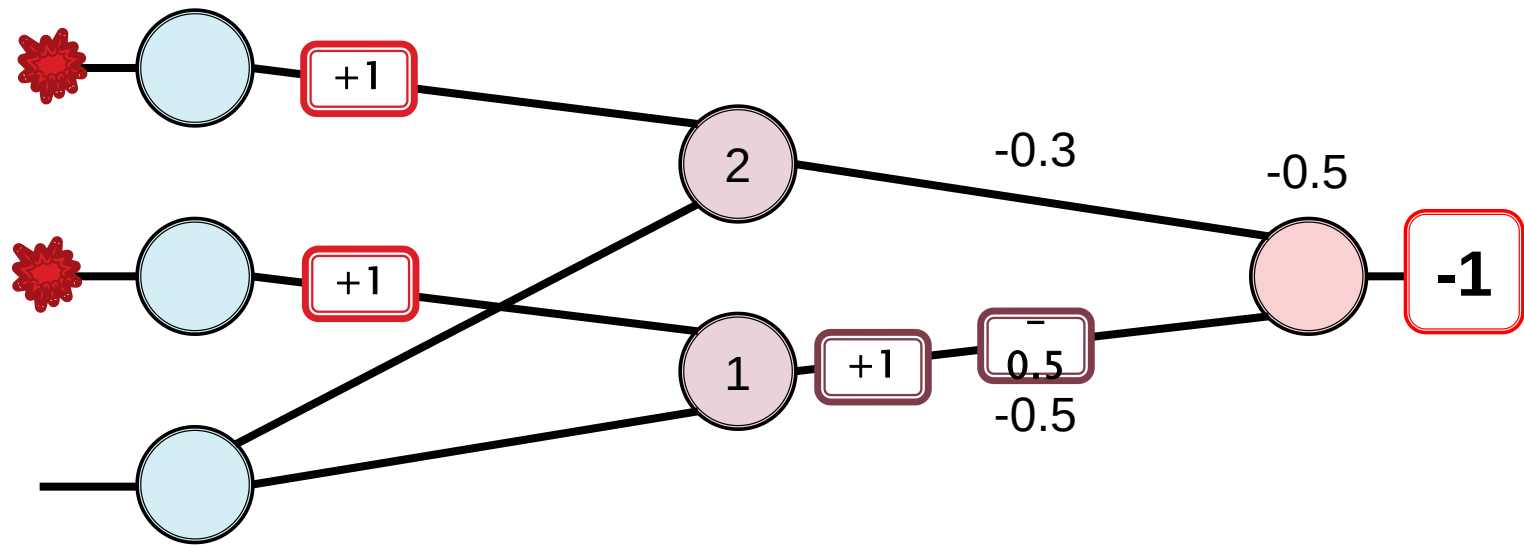
Простым R-элементом (Реагирующим элементом) называется элемент, который выдаёт выходной сигнал +1 , если сумма его входных сигналов является строго положительной, и сигнал -1 , если сумма его входных сигналов является строго отрицательной. Если сумма входных сигналов равна нулю, выход можно считать либо равным нулю, либо неопределённым.

Пример работы персептрона



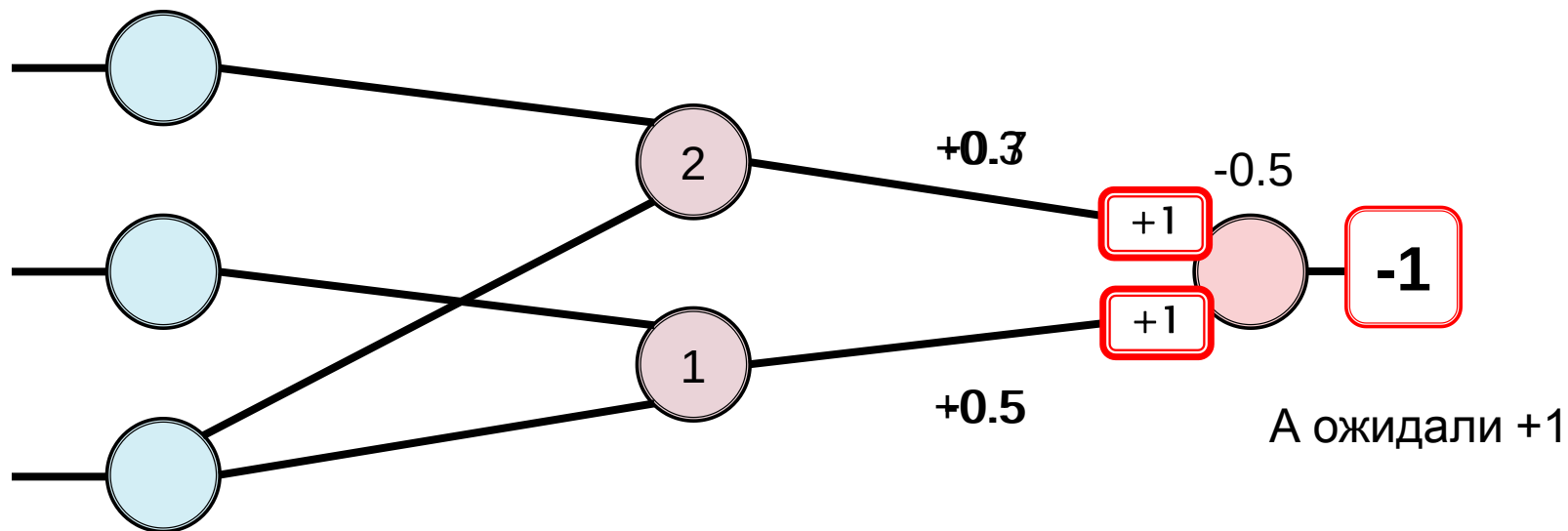
Пример 1

Пример работы персептрона



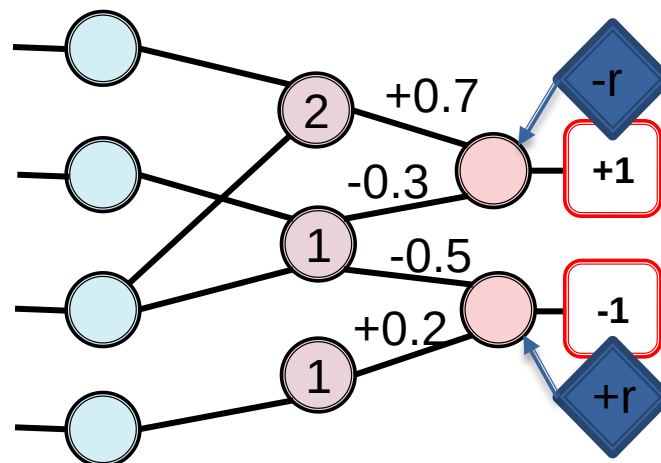
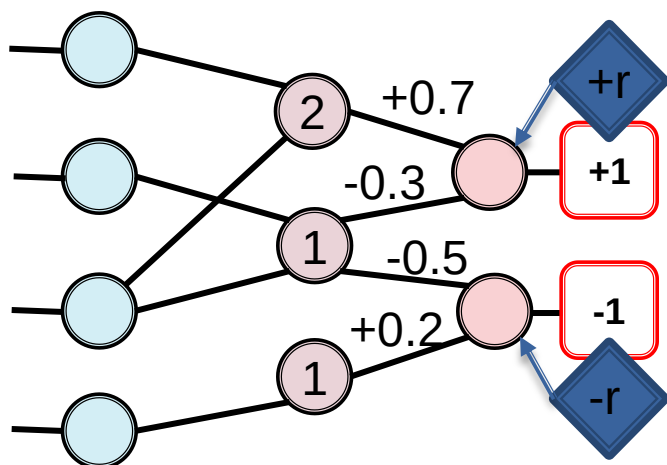
Пример 2

Алгоритмы обучения персептронов



Классический метод обучения персептрона — это обучение с коррекцией ошибки. Представляет собой такой метод обучения, при котором вес связи не изменяется до тех пор, пока текущая реакция персептрона остается правильной. При появлении неправильной реакции вес изменяется на единицу, а знак (+/-) определяется противоположным от знака ошибки.

Алгоритмы обучения персептронов



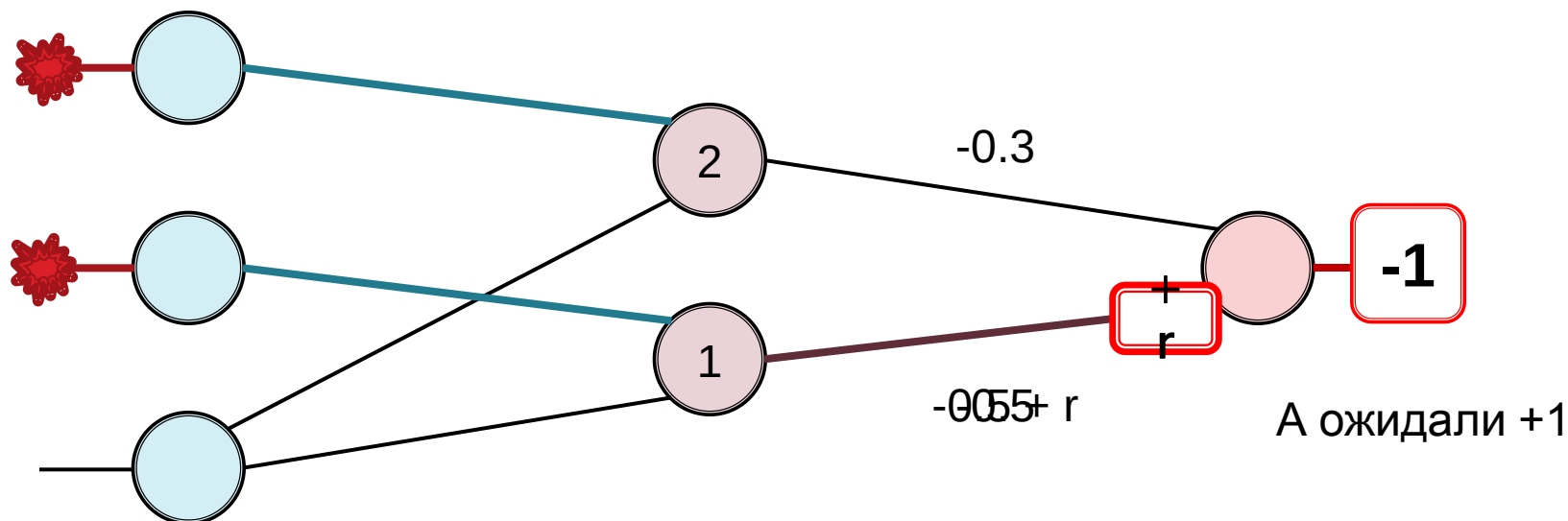
Кроме того, Розенблатт пытался классифицировать различные алгоритмы обучения, называя их системами подкрепления.

Он различал следующие виды:

Положительное подкрепление — представляет собой такой процесс подкрепления, при котором вес связи, начинающийся на активном элементе u_i и оканчивающийся на элементе u_j , изменяется на величину r , знак которой совпадает со знаком сигнала u_j^* .

Отрицательное подкрепление — представляет собой такой процесс подкрепления, при котором вес связи, начинающийся на активном элементе u_i и оканчивающийся на элементе u_j , изменяется на величину r , знак которой противоположен знаку сигнала u_j^* .

Алгоритмы обучения персептронов



Кроме, классического метода обучения персептрона Розенблатт также ввел понятие об обучении без учителя, предложив следующий способ обучения: **Альфа** — системой подкрепления называется система подкрепления, при которой веса всех активных связей c_{ij} , изменяются на одинаковую величину r , а веса не активных связей за это время не изменяются.

Методы обучения синаптических весов нейрона

Персептрон Розенблатта (1957)

Зададим веса случайным образом.

Подадим на вход модели МакКаллока-Питтса все обучающие объекты x_i по очереди и выданные ответы сравним с правильными, т.е. используем так называемое

ОБУЧЕНИЕ С УЧИТЕЛЕМ.

Если $a(x_i) = y_i$, то веса не меняем. Если $a(x_i) = 0, y_i = 1$, то вектор весов увеличиваем (**правило Хэбба**)
$$w = w + \eta x_i,$$

$\eta > 0$ - темп обучения (learning rate).

Если $a(x_i) = 1, y_i = 0$, то уменьшаем веса $w = w - \eta x_i$.

Все три случая можно объединить в один $w = w - \eta (a(x_i) - y_i) x_i$.

Обучающие объекты проходят через это правило многократно, до тех пор, пока веса изменяются.

Алгоритм обучения персептрона Розенблатта

Вход:

X^l - обучающая выборка

η - темп обучения

Выход:

- синаптические веса $w_0, w_1, \dots, w_n$

-
1. инициализировать веса w_j
 2. повторять
 3. **для всех** $i=1, \dots, l$
 4. $w = w - \eta(a(x_i) - y_i)x_i$;
 5. пока веса изменяются.
-

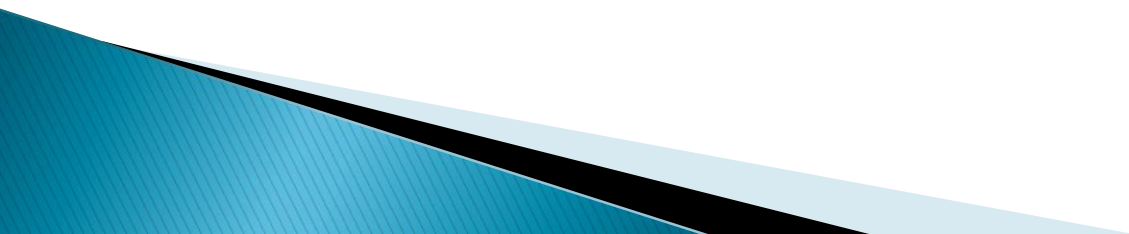
Заключение

Описывая эти системы подкрепления Розенблатт основывался на идеях Хебба.

Затем с появлением многослойного персептрона обучение с коррекцией ошибки было модифицировано и его стали называть дельта-правило.

Модификация была проведена с целью сделать функцию обучения дифференцированной, что в свою очередь нужно для применения метода градиентного спуска, благодаря которому и возможно обучение более одного слоя. При этом пришлось отказаться от бинарного сигнала, и пользоваться на входе вещественными числами.

Подробнее об этом расскажет правило Хебба.

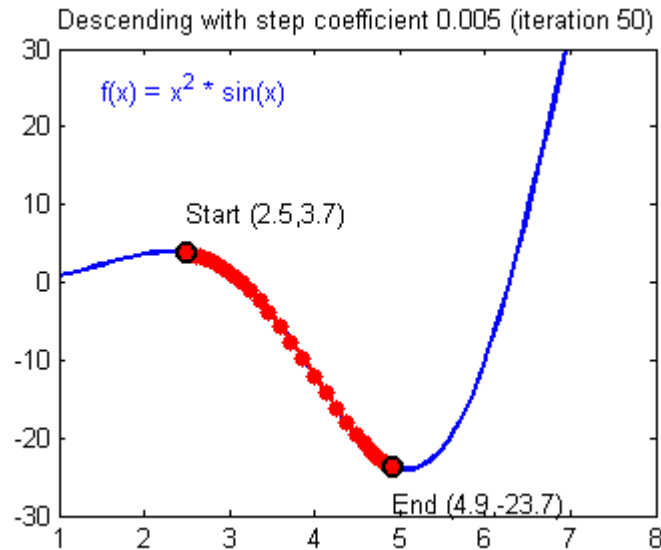


Правило Хебба



Дональд Олдинг Хебб — канадский физиолог и нейропсихолог. Известен работами, приведшими к пониманию значения нейронов для процесса обучения. Его также называют одним из создателей теории искусственных нейронных сетей, так как он предложил первый работающий алгоритм обучения искусственных нейронных сетей.

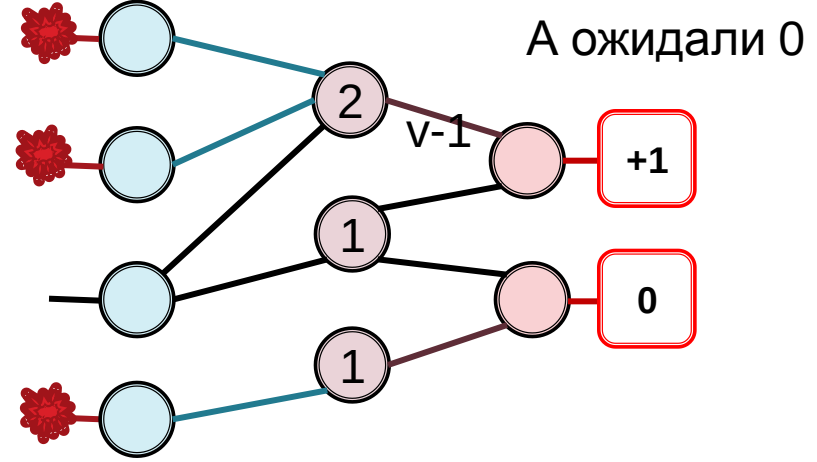
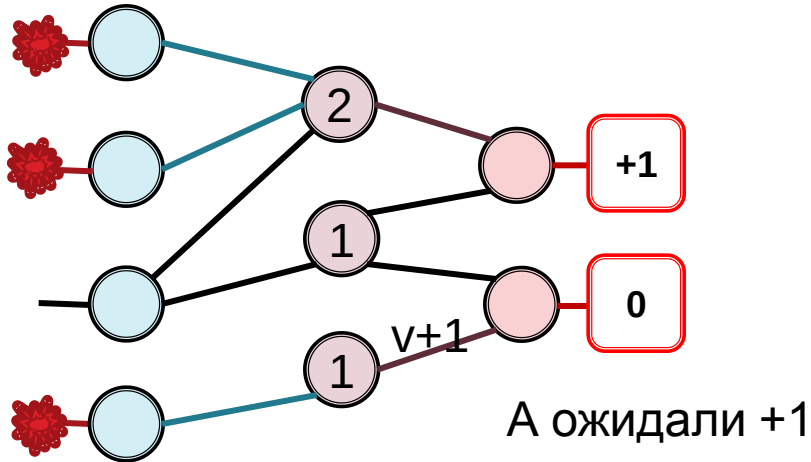
Правило Хебба



Правило Хебба есть не что иное, как реализация *метода градиентного спуска по поверхности ошибки. По сути, мы заставляем нейрон подстроиться под подаваемые сигналы, сдвигая каждый раз его веса в сторону, противоположную ошибке, то есть в направлении антиградиента. Чтобы градиентный спуск привел нас к локальному экстремуму, не проскочив его, скорость спуска должна быть достаточно мала. Что в Хеббовском обучении учитывается малостью параметра λ .

*(метода градиентного спуска – 2 лабораторная у Геренштейна)

Правило Хебба



Правила корректировки весов предложены Хеббом в 1949 году и имеют следующий смысл:

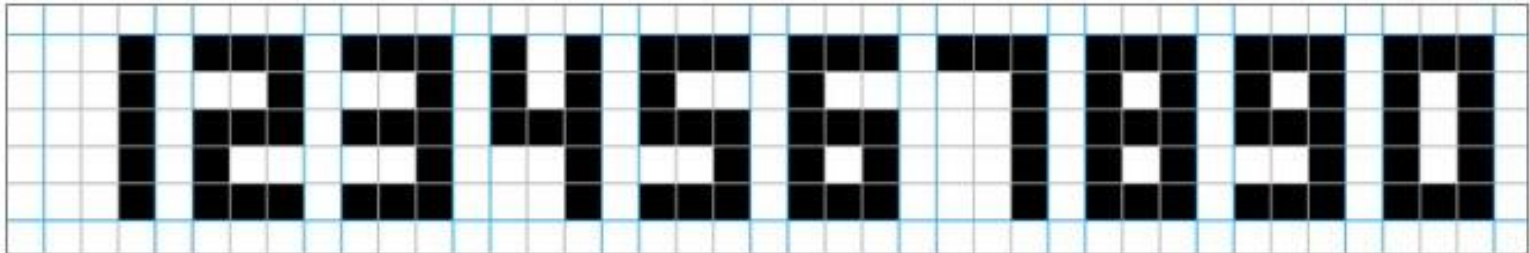
- **Первое правило Хебба** — Если сигнал персептрона неверен и равен нулю, то необходимо увеличить веса тех входов, на которые была подана единица.
- **Второе правило Хебба** — Если сигнал персептрона неверен и равен единице, то необходимо уменьшить веса тех входов, на которые была подана единица.

Нейросеть распознавание цифр по правилу Хебба

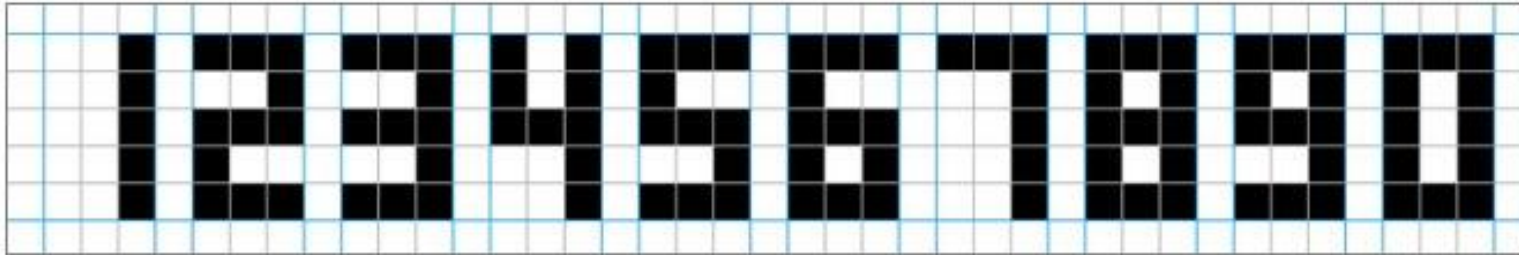
Постановка задачи

Программа из 10 цифр будет распознавать нужную нам цифру. Например, пусть это будет цифра 5.

Нашей обучающей выборкой будут все цифры от 0 до 9.



Нейросеть распознавание цифр по правилу Хебба

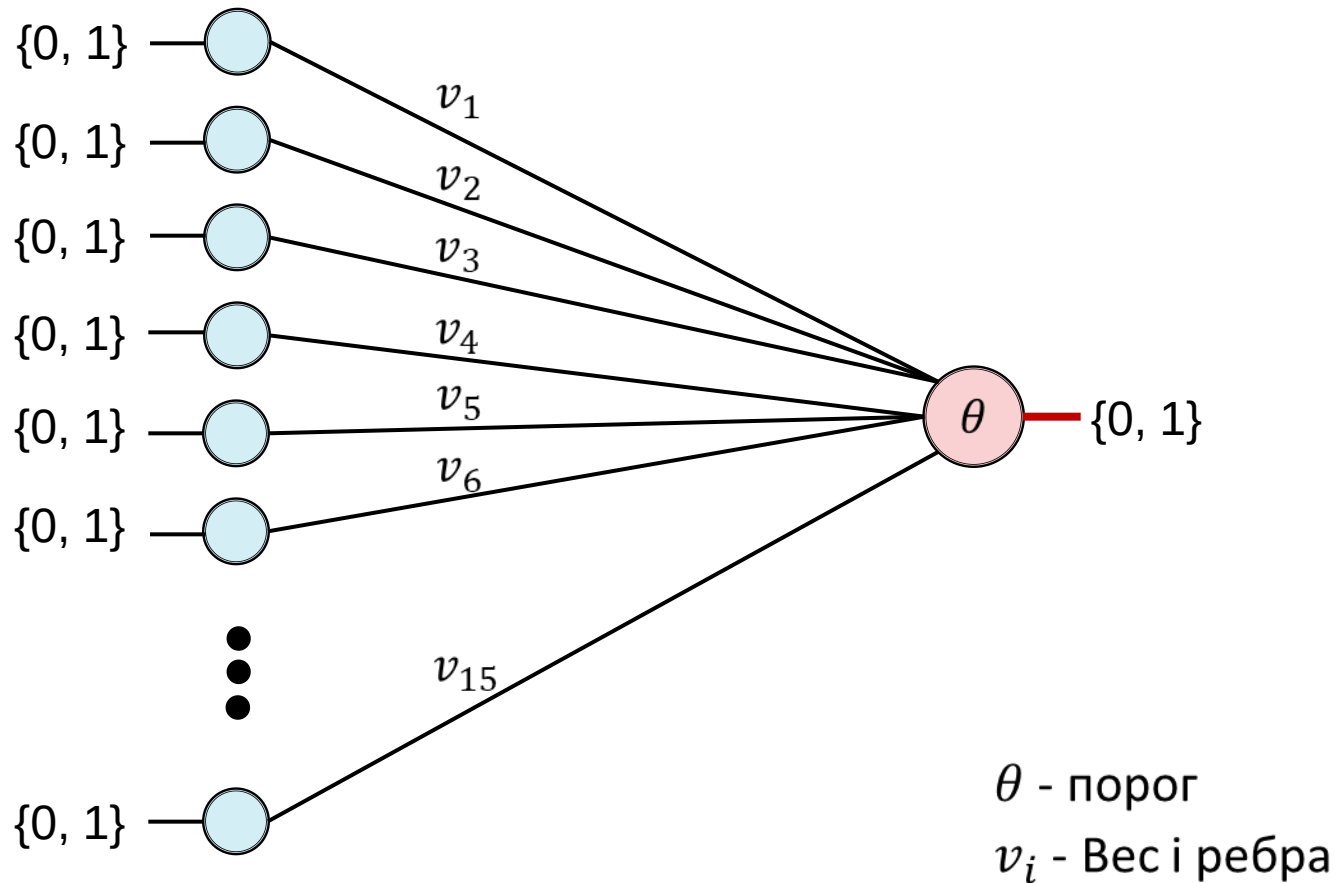


0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	1	0	0	1	0	0	1	1	0	0	1	1	0
0	0	1	1	1	1	1	1	1	0	0	1	1	1	1
0	0	1	1	0	0	0	1	0	0	1	1	0	0	1
0	0	1	1	1	1	0	0	1	1	1	1	1	1	1

Чтобы работать с нейросетью, мы должны на ее входы подавать сигналы в виде чисел (0 или 1). Таким образом изображение цифры мы должны перевести в последовательность сигналов в виде чисел. Это легко сделать, если представить цифры в строковом формате.

Каждая цифра представляет собой всего пятнадцать квадратиков, причем только двух возможных цветов. За белый квадратик отвечает 0, а черный квадратик – 1. Поэтому наши десять цифр от 0 до 9 в строковом формате будут выглядеть вот так.

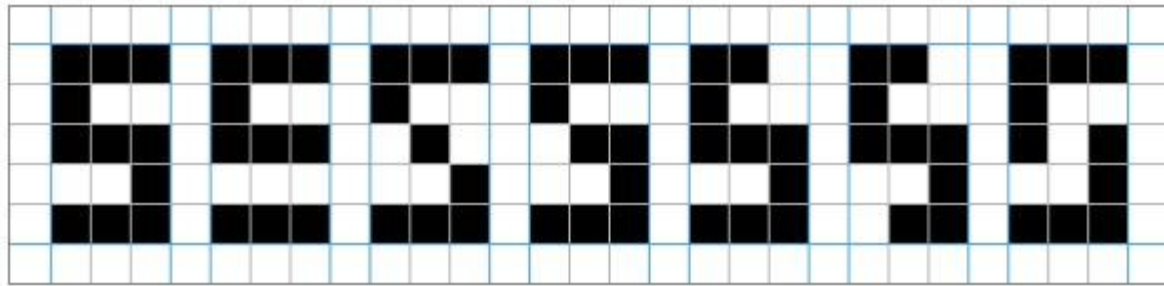
Нейросеть распознавание цифр по правилу Хебба



В нашей сети будет по 1 S-элементу (он же A-элемент) на каждый квадратик из таблички. Поэтому для распознавания цифры нам потребуется 15 сенсоров.

Нейросеть распознавание цифр по правилу Хебба

Когда нейросеть обучиться безошибочно распознавать нужную нам цифру (5), тогда мы проверим ее «интеллект» уже на тестовой выборке. Эта выборка будет похитрее: на вход будут подаваться уже искаженные изображения пятерки.



Нейросеть распознавание цифр по правилу Хебба

Алгоритм обучения

Мы будем случайным образом выбирать цифру и прогонять ее через сеть, модифицируя ее веса.

Важность тем или иным S-элементам придают веса, связывающие их с R-элементом. Таким образом, по правилам Хебба:

- Если наша нейросеть правильно распознала/отвергла цифру 5, то мы ничего не предпринимаем (все ведь замечательно!).
- Если нейросеть ошиблась и распознала неверную цифру как 5, то мы должны ее наказать – мы уменьшаем веса тех связей, через которые прошел сигнал. Другими словами веса, связанные с возбуждившимися входами, уменьшаются.
- Если нейросеть ошиблась и не распознала цифру 5, то мы должны увеличить все веса, через которые прошел сигнал. Таким образом мы как бы говорим сети, что такие связи, а значит и связанные с ними входы – правильные.

Нейросеть распознавание цифр по правилу Хебба

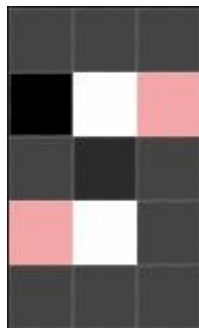
После обучения

Получим следующие веса:

Более темные квадраты уверяют нашу суеверную нейросеть, что предлагаемая цифра несомненно является пятеркой (3 и 2).

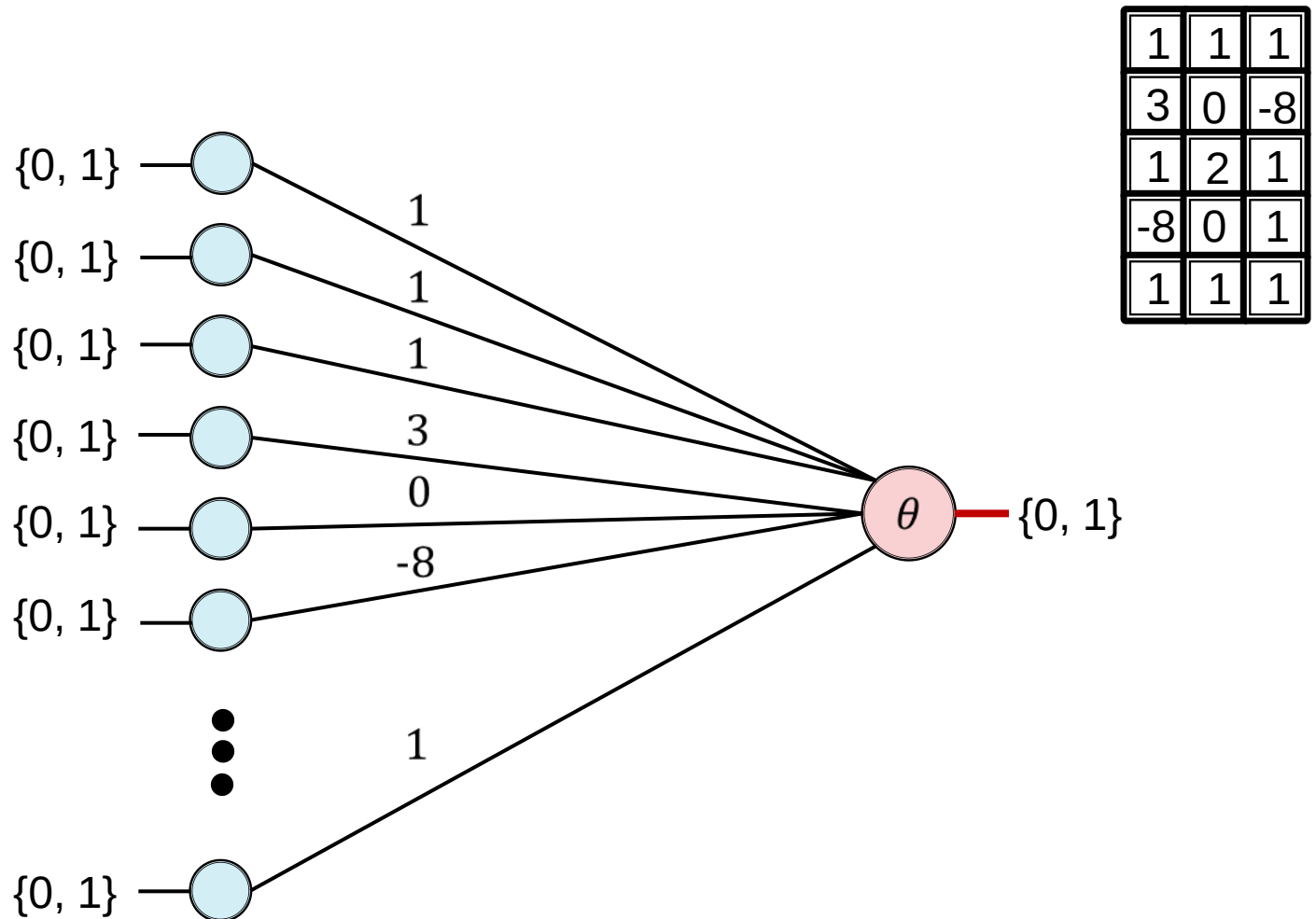
Серые области вносят обычный вклад (по 1).

Красные квадратики вызывают отвращение у сети и их появление на входах приводит аж к отрицательным значениям (-8).



1	1	1
3	0	-8
1	2	1
-8	0	1
1	1	1

Нейросеть распознавание цифр по правилу Хебба



θ - порог

v_i - Вес i ребра

Правило Хэбба

Иногда удобнее полагать, что классы помечены числами -1 и 1 , а нейрон выдает знак скалярного произведения

$$a(x) = \text{sign}(\langle w, x \rangle)$$

Тогда несовпадение знаков означает, что нейрон ошибается на объекте x_i при этом пункт (4) алгоритма преобразуется в следующее правило модификации весов

$$\langle w, x_i \rangle y_i < 0, \text{ то } w = w + \eta x_i y_i$$

Метод стохастического градиента

Рассмотрим задачу настройки (обучения) синаптических весов в модели МакКаллока-Питтса. Пусть $y^*: X \rightarrow Y$ - целевая зависимость, известная только на объектах обучающей выборки

$$X^l = (x_i, y_i)_{i=1}^l, \quad y_i = y^*(x_i).$$

Требуется найти вектор w , доставляющий минимум функционалу качества

$$Q(w) = \sum_{i=1}^l L(a(x_i, w), y_i) \rightarrow \min_w,$$

где $L(a, y)$ - заданная функция потерь, характеризующая величину ошибки ответа a при правильном ответе y .

Для минимизации $Q(w)$ применим метод градиентного спуска. В этом методе выбирается некоторое начальное приближение для вектора весов w , затем запускается итерационный процесс, на каждом шаге которого вектор w изменяется в направлении наиболее быстрого убывания функционала $Q(w)$:

$$w = w - \eta \nabla Q(w),$$

где $\nabla Q(w) = \left(\frac{\partial Q(w)}{\partial w_j} \right)_{j=1}^n$ - вектор градиента, $\eta > 0$ - величина шага в направлении антиградиента (темп обучения).

Метод стохастического градиента

Предполагая, что функция потерь $L(a, y)$ и функция активации φ дифференцируемы, распишем градиент:

$$w = w - \eta \sum_{i=1}^l L'_a(a(x_i, w), y_i) \varphi'(\langle w, x_i \rangle) x_i.$$

Каждая пара (x_i, y_i) вносит аддитивный вклад в изменение вектора w , НО вектор w изменяется только после перебора всех l объектов. Оказывается, незначительная модификация этого процесса способна существенно повысить скорость сходимости.

Для этого надо выбирать пары (x_i, y_i) по одной, для каждой делать градиентный шаг и сразу обновлять вектор весов:

$$w = w - \eta L'_a(a(x_i, w), y_i) \varphi'(\langle w, x_i \rangle) x_i.$$

В методе стохастического градиента (stochastic gradient, SG) пары (x_i, y_i) перебираются в случайном порядке. Если же объекты предъявлять в некотором фиксированном порядке, процесс чаще оказывается расходящимся или заикливаясь.

Алгоритм обучения методом SG

Вход:

X^l - обучающая выборка

η - темп обучения

λ - параметр сглаживания функционала Q

Выход:

- синаптические веса $w_0, w_1, \dots, w_n$

-
1. инициализировать веса w_j
 2. инициализировать текущую оценку функционала: $Q = \sum_{i=1}^l L(a(x_i, w), y_i)$
 3. **повторять**
 4. выбрать объект x_i из X^l (случайно)
 5. вычислить выходное значение алгоритма $a(x_i, w)$ и ошибку:
$$\varepsilon_i = L(a(x_i, w), y_i)$$
 6. сделать шаг градиентного спуска:
$$w = w - \eta L'_a(a(x_i, w), y_i) \varphi'(\langle w, x_i \rangle) x_i$$
 7. оценить значение функционала:
$$Q = (1 - \lambda)Q + \lambda \varepsilon_i$$
 8. **пока** значение Q не стабилизируется и/или веса w не перестанут изменяться.
-

Частный случай: Дельта-правило ADALINE

Задача регрессии: Пусть $X = R^{n+1}$, $Y \subseteq R$.

$$L(a, y) = (a - y)^2.$$

Функция активации - линейная $\varphi(z) = z$. Нейронная сеть с такой функцией активации называется **ADALINE** (Adaptive Linear Element).

Градиентный шаг - **дельта-правило** (delta-rule) (Видроу-Хофф)

$$w = w - \eta \underbrace{(a(x_i, w) - y_i)}_{\Delta_i} x_i$$

Δ_i - ошибка алгоритма на объекте x_i .

В силу линейности функции активации: $a(x_i, w) = \langle w, x_i \rangle$, тогда

$$w = w - \eta (\langle w, x_i \rangle - y_i) x_i$$

Алгоритм метода Видроу-Хоффа

Вход:

X^l - обучающая выборка

η - темп обучения

Выход:

- синаптические веса $w_0, w_1, \dots, w_n$

-
1. инициализировать веса w_j
 2. повторять
 3. **для всех** $i=1, \dots, l$
 4. $w = w - \eta (\langle w, x_i \rangle - y_i) x_i;$
 5. пока веса изменяются.
-

Квадратичная функция ошибки для однослойной нейронной сети (2)

- Квадратичная функции ошибки (используется для задачи восстановления регрессии):

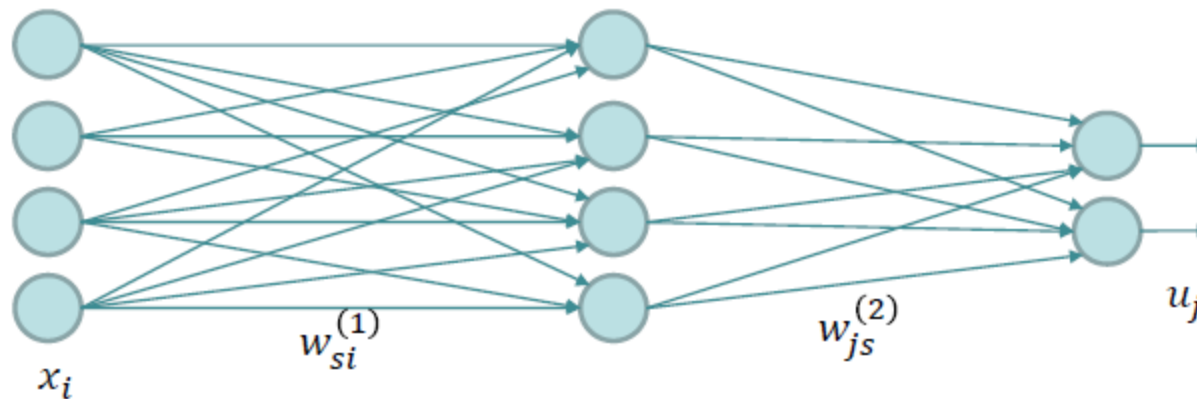
$$\begin{aligned} E &= \frac{1}{2} \sum_{k=1}^L \|y^k - u^k\|^2 = \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M (y_j^k - u_j^k)^2 \\ &= \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M \left(y_j^k - \varphi \left(\sum_{i=0}^N w_{ji} x_i^k \right) \right)^2, \end{aligned}$$

где $y^k = (y_j^k)_{j=\overline{1,M}} \in Y$ – множество обучающих примеров, а $u^k = (u_j^k)_{j=\overline{1,M}}$ – выход сети для входа $x^k = (x_i^k)_{i=\overline{1,N}} \in X$

- Также вводится нормализация указанной метрики по числу примеров обучающей выборки L

Квадратичная функция ошибки для двухслойной нейронной сети (1)

- Рассмотрим двухслойную нейронную сеть:



- $w_{si}^{(1)}, w_{js}^{(2)}$ – веса синаптических связей
- Выходной сигнал нейрона скрытого слоя описывается следующим образом:
$$v_s = \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right), s = \overline{0, K},$$
 где K – количество нейронов на скрытом слое

□ Сигнал j -ого нейрона выходного слоя:

$$u_j = \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} v_s \right) = \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i \right) \right), j = \overline{1, M}$$

□ Квадратичная функция ошибки для набора тренировочных примеров имеет вид:

$$\begin{aligned} E(w) &= \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M (y_j^k - u_j^k)^2 \\ &= \frac{1}{2} \sum_{k=1}^L \sum_{j=1}^M \left(y_j^k - \varphi^{(2)} \left(\sum_{s=0}^K w_{js}^{(2)} \varphi^{(1)} \left(\sum_{i=0}^N w_{si}^{(1)} x_i^k \right) \right) \right)^2 \end{aligned}$$